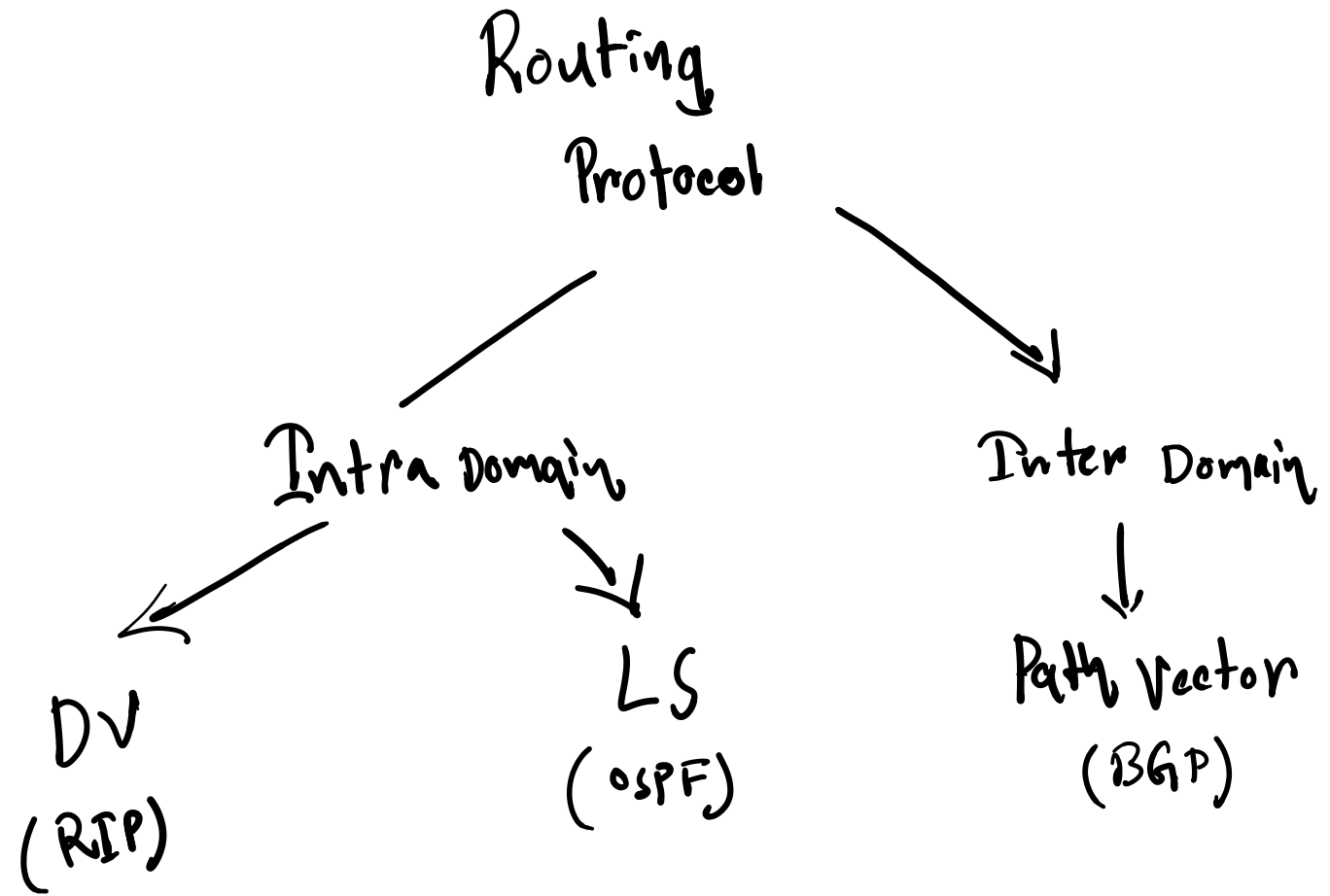


Network Layer: Routing Protocols



9th edition
Jim Kurose, Keith Ross
Pearson, 2025

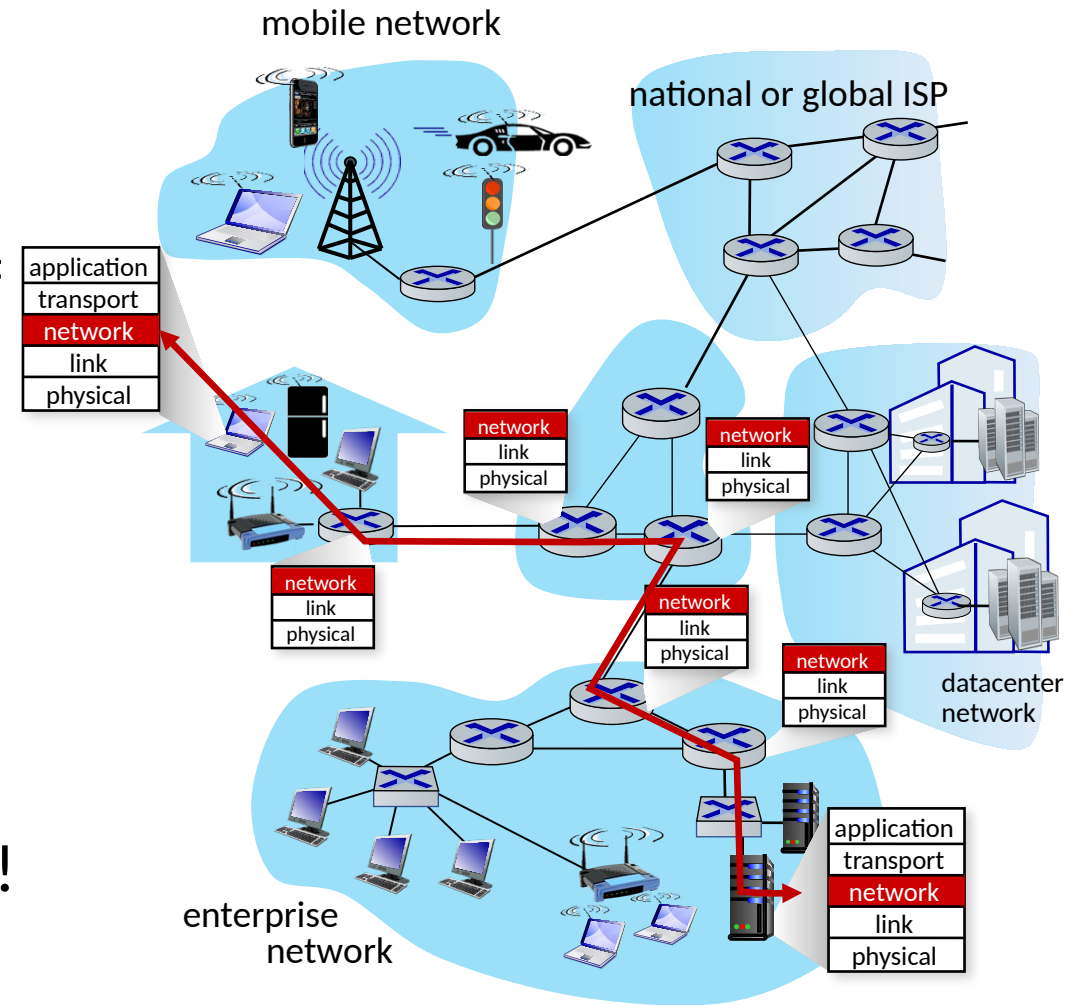
Routing protocols

Routing protocol goal: determine “good” paths (equivalently, routes), from sending hosts to receiving host, through network of routers

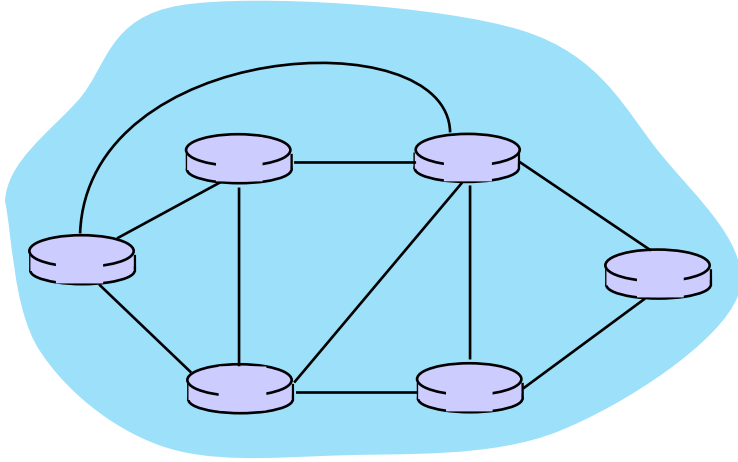
path: sequence of routers packets traverse from given initial source host to final destination host

“good”: least “cost”, “fastest”, “least congested”

routing: a “top-10” networking challenge!



Graph abstraction: link costs



cost of link connecting and

$$c_{ij} = 5, \quad c_{ji} = \infty$$

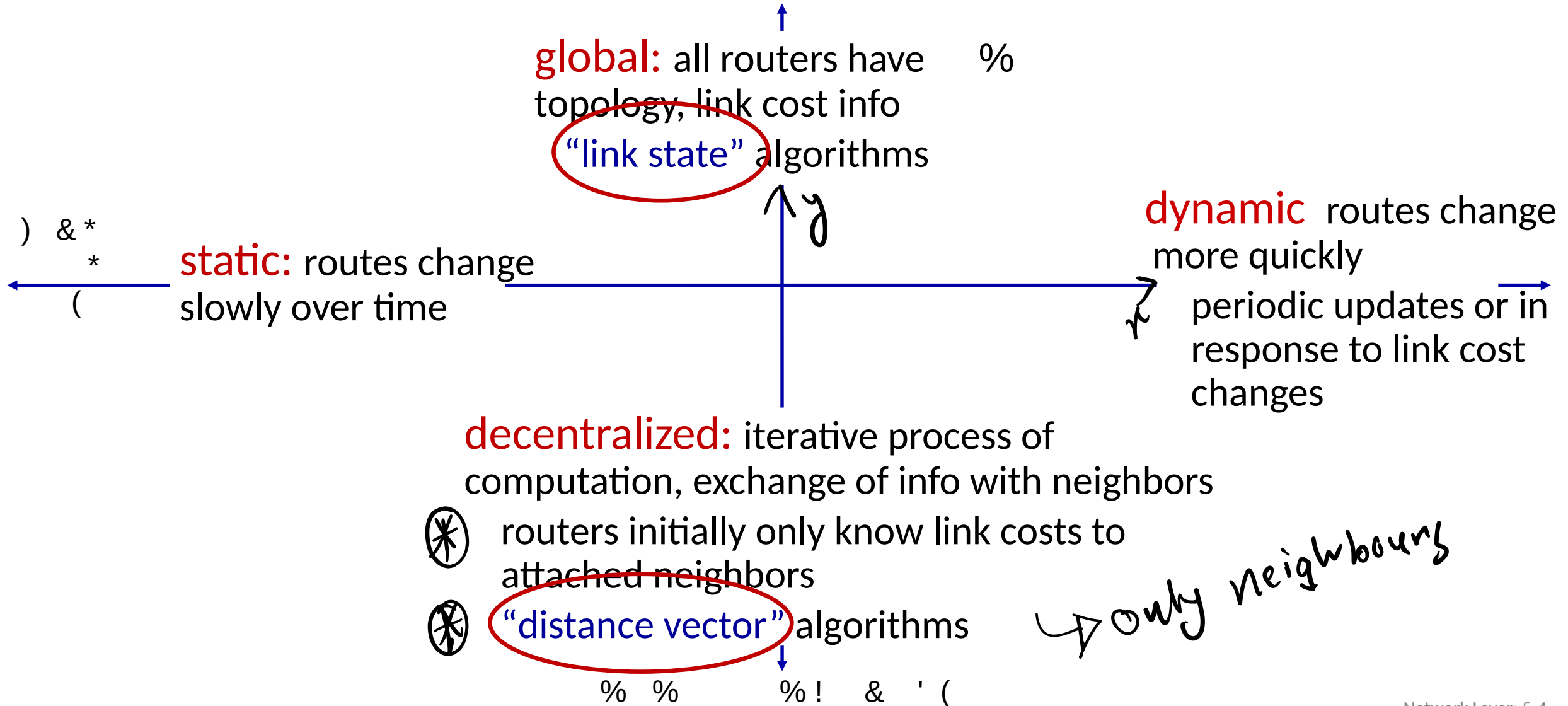
cost defined by network operator:
could always be 1, or inversely related
to bandwidth, or inversely related to
congestion

graph:

set of routers = { " # \$! }

set of links = { (" # " # " # \$ \$! \$!) }

Routing algorithm classification



1 Centralized vs Decentralized Routing Algorithms

Routing algorithms decide the best (least-cost) path from source to destination.

♦ Centralized Routing Algorithm (Link-State)

📌 Core Idea

- One algorithm has complete, global knowledge of the entire network.
- Knows:
 - All routers
 - All links
 - Cost of every link

🧠 How it works

1. The algorithm collects full network topology information.
2. It computes the shortest path (usually using Dijkstra's algorithm).
3. Results are either:
 - Computed at a central controller (SDN style), or
 - Computed independently by every router using the same full information.

📡 Why called Link-State (LS)

- Each router knows the state (cost, status) of every link.
- Routers flood link-state information throughout the network.

📘 Example

- OSPF
- IS-IS

✅ Advantages

- Fast convergence
- Accurate routing
- Less chance of routing loops

❌ Disadvantages

- Higher overhead (flooding)
- More memory and CPU usage

♦ Decentralized Routing Algorithm (Distance Vector)

📌 Core Idea

- No router has full network knowledge.
- Each router knows:
 - Cost to its direct neighbors only

🧠 How it works

1. Routers exchange distance information with neighbors.
2. Each router updates its routing table iteratively.
3. Over time, routes converge.

📡 Why called Distance-Vector (DV)

- Each router maintains a vector of distances to all destinations.

📘 Example

- RIP

✅ Advantages

- Simple
- Low computational overhead

❌ Disadvantages

- Slow convergence
- Routing loops (count-to-infinity problem)

🔍 Key Difference Summary

Feature	Link-State	Distance-Vector
Knowledge	Global	Local
Algorithm	Dijkstra	Bellman-Ford
Convergence	Fast	Slow
Overhead	Higher	Lower
Example	OSPF	RIP



2 Static vs Dynamic Routing Algorithms

♦ Static Routing

📌 Characteristics

- Routes change very slowly
- Often configured manually
- No automatic adaptation to failures

■ Example

- Manually configured routes in routers

✅ Advantages

- Simple
- No routing overhead

❌ Disadvantages

- Not scalable
 - Cannot adapt to failures
-

♦ Dynamic Routing

📌 Characteristics

- Routes change automatically
- React to:
 - Link failures
 - Topology changes
 - Network conditions

■ Example

- RIP
- OSPF
- BGP

⚠️ Problems

- Routing loops
 - Route oscillation
 - Control traffic overhead
-

3 Load-Sensitive vs Load-Insensitive Routing

♦ Load-Sensitive Routing

📌 Core Idea

- Link cost changes based on congestion ✎
- Congested links get higher cost
- Traffic is routed around congestion

■ Historical Example

- Early ARPANET ✎

❌ Problems Found

- Rapid cost fluctuations
 - Route oscillation
 - Network instability
-

♦ Load-Insensitive Routing (Modern Internet)

📌 Core Idea

- Link costs are fixed or manually configured
- Cost does NOT depend on current traffic load

■ Examples

- RIP
- OSPF
- BGP

✅ Why preferred

- Stable routing
 - Predictable behavior
 - Less oscillation
-

Network layer: “control plane” roadmap

- introduction
- routing protocols
 - **link state** → messaging type
 - distance vector
- intra-ISP routing: OSPF
- routing among ISPs: BGP
- SDN control plane
- Internet Control Message Protocol

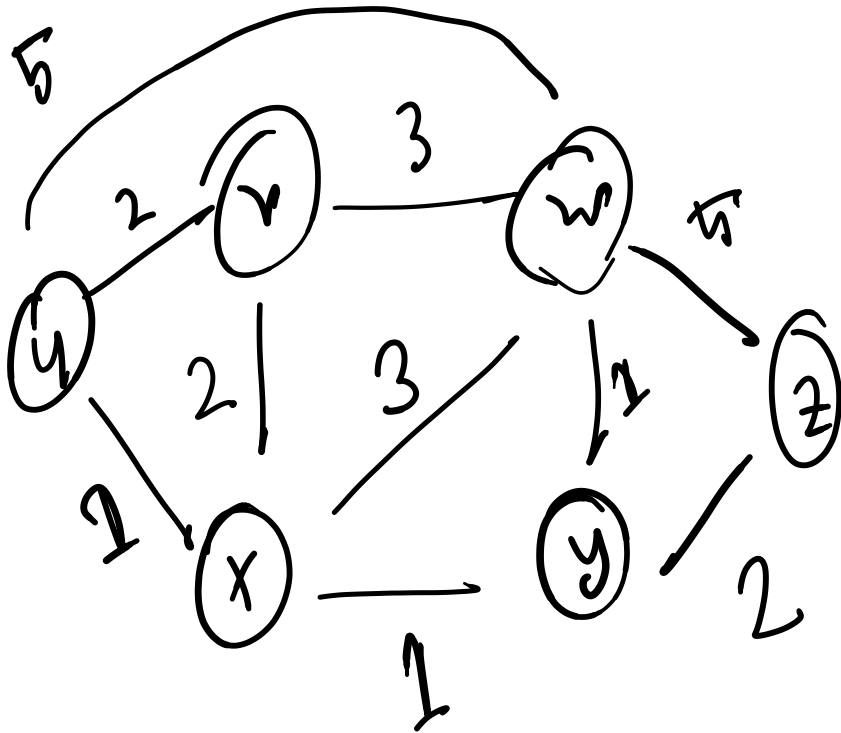


- network management, configuration
 - SNMP
 - NETCONF/YANG

Global ↘

u-x-y-z

link state → uses Dijkstra



cost → congestion

Dijkstra's link-state routing algorithm

centralized: network topology, link costs known to *all* nodes

accomplished via “link state broadcast”

all nodes have same info

computes least cost paths from one node (“source”) to all other nodes

gives forwarding table for that node

iterative: after k iterations, know least cost path to k destinations

notation

$c_{x,y}$: direct link cost from node x to y ; $= \infty$ if not direct neighbors

$D(v)$: *current* estimate of cost of least-cost-path from source to destination v

$p(v)$: predecessor node along path from source to v

N' : set of nodes whose least-cost-path *definitively* known

Dijkstra's link-state routing algorithm

1 *Initialization:*

2 $N' = \{u\}$

3 for all nodes v

4 if v adjacent to u /* u initially knows direct-path-cost only to direct neighbors */

5 then $D(v) = c_{u,v}$ /* but may not be *minimum* cost! */

6 else $D(v) = \infty$

7



8 *Loop*

9 find w not in N' such that $D(w)$ is a minimum

10 add w to N'

11 update $D(v)$ for all v adjacent to w and not in N' :

12

$$d_v \Rightarrow \min(d_v, d_w + c_{wv})$$

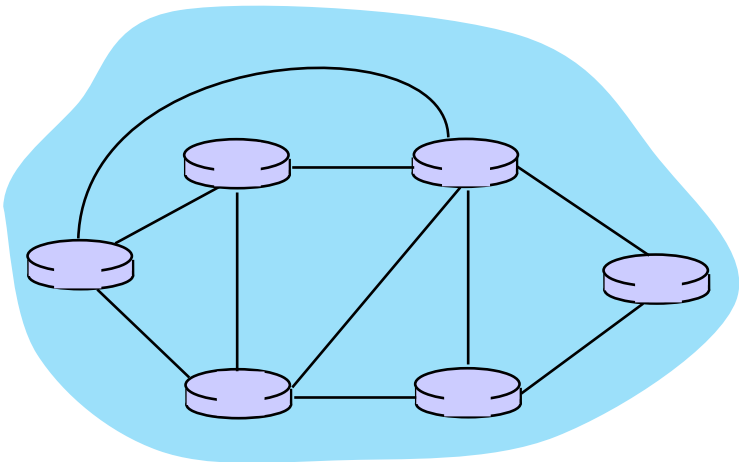
13 /* new least-path-cost to v is either old least-cost-path to v or known

14 least-cost-path to w plus direct-cost from w to v */

until all nodes in N'

Dijkstra's algorithm: an example

	V	W	X	Y	Z
!	"# \$% # \$	"# \$% # \$	"# \$% # \$	"# \$% # \$	"# \$% # \$
	%	%	%		

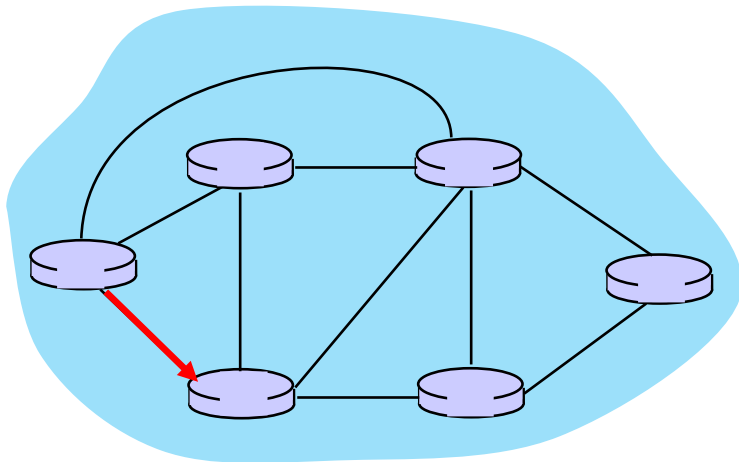


Initialization (step 0):

For all a : if a adjacent to u then $D(a) = c_{u,a}$

Dijkstra's algorithm: an example

	V	W	X	Y	Z
!	"# \$% # \$	"# \$% # \$	"# \$% # \$	"# \$% # \$	"# \$% # \$
	%	%	%		



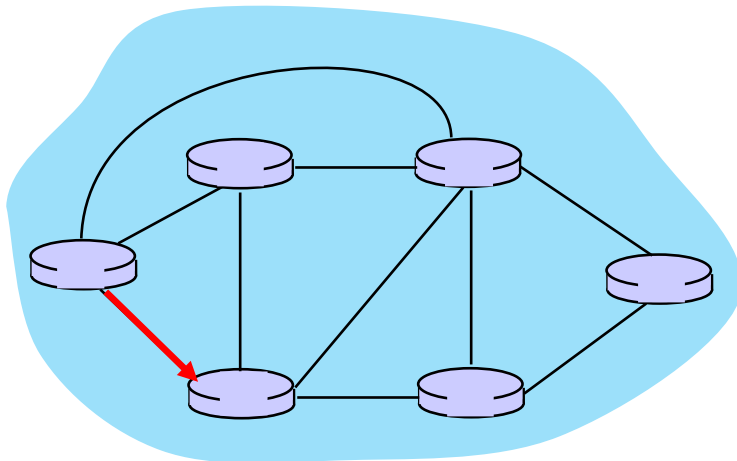
8 *Loop*

9 find a not in N' such that $D(a)$ is a minimum

10 add a to N'

Dijkstra's algorithm: an example

	V	W	X	Y	Z
!	"# \$% # \$	"# \$% # \$	"# \$% # \$	"# \$% # \$	"# \$% # \$
	%	%	%		
	%	%		%	



8 Loop

- 9 find a not in N' such that $D(a)$ is a minimum
- 10 add a to N'
- 11 update $D(b)$ for all b adjacent to a and not in N' :

$$D(v) = \min (D(v), D(x) + c_{x,v}) = \min(2, 1+2) = 2$$

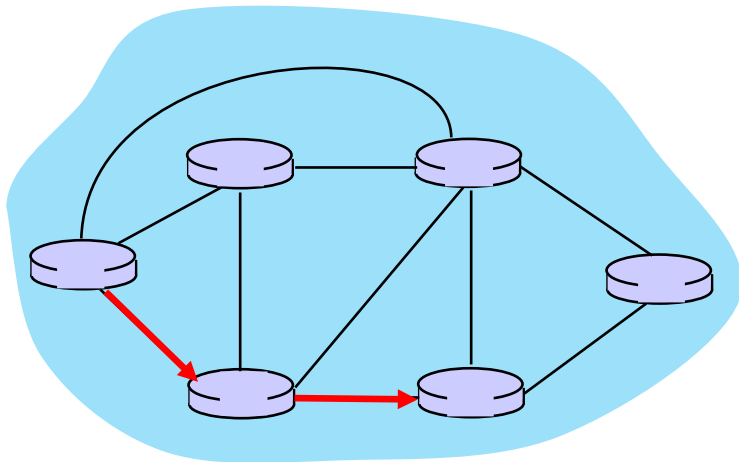
$$D(w) = \min (D(w), D(x) + c_{x,w}) = \min (5, 1+3) = 4$$

$$D(y) = \min (D(y), D(x) + c_{x,y}) = \min(\text{inf}, 1+1) = 2$$

NEW!

Dijkstra's algorithm: an example

	V	W	X	Y	Z
!	"# \$% # \$	"# \$% # \$	"# \$% # \$	"# \$% # \$	"# \$% # \$
	%	%	%		
	%	%		%	

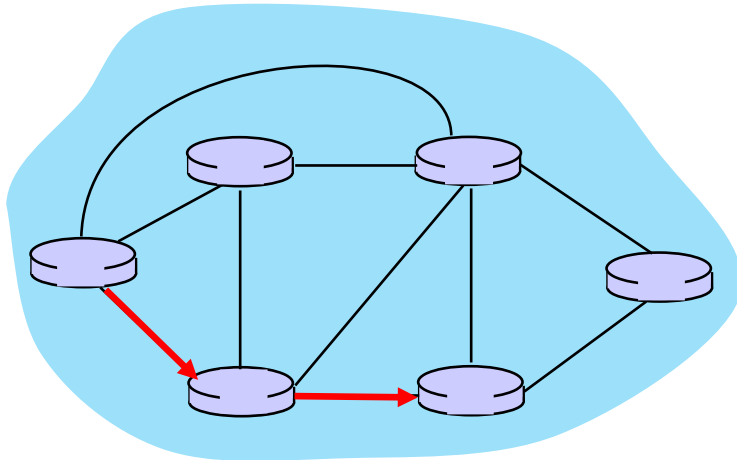


8 Loop

- 9 find a not in N' such that $D(a)$ is a minimum
- 10 add a to N'

Dijkstra's algorithm: an example

	V	W	X	Y	Z
!	"# \$% # \$	"# \$% # \$	"# \$% # \$	"# \$% # \$	"# \$% # \$
	%	%	%		
	%	%		%	
	%	%			%



8 Loop

- 9 find a not in N' such that $D(a)$ is a minimum
- 10 add a to N'
- 11 update $D(b)$ for all b adjacent to a and not in N' :

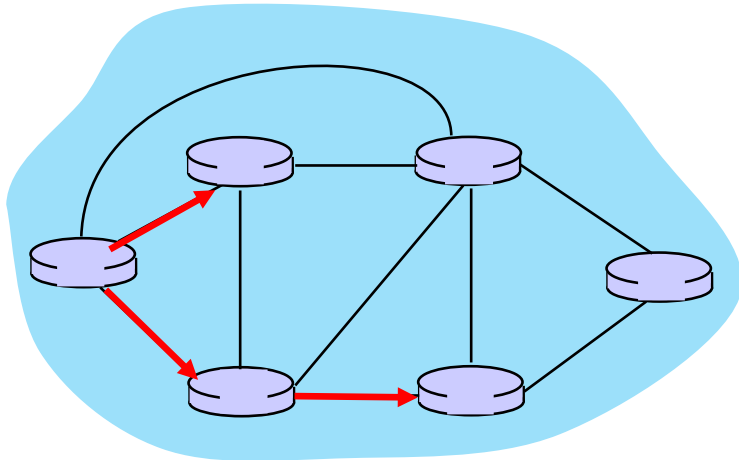
$$D(w) = \min (D(w), D(y) + c_{y,w}) = \min (4, 2+1) = 3$$

$$D(z) = \min (D(z), D(y) + c_{y,z}) = \min (\text{inf}, 2+2) = 4$$



Dijkstra's algorithm: an example

	V	W	X	Y	Z
!	"# \$% # \$	"# \$% # \$	"# \$% # \$	"# \$% # \$	"# \$% # \$
	%	%	%		
	%	%		%	
	%	%			%

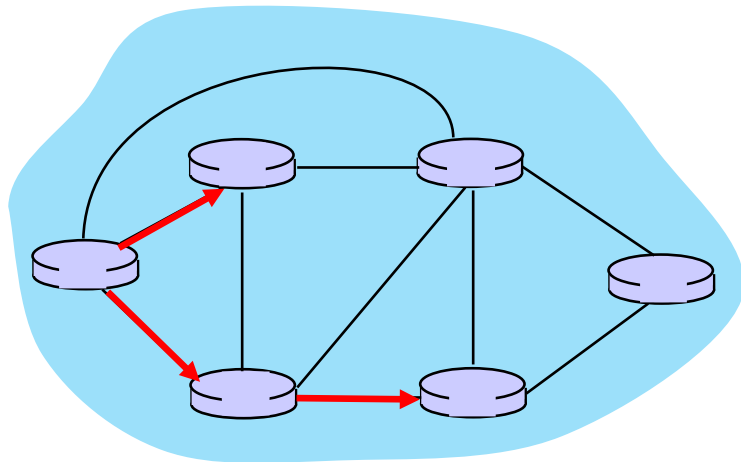


8 Loop

- 9 find a not in N' such that $D(a)$ is a minimum
- 10 add a to N'

Dijkstra's algorithm: an example

	V	W	X	Y	Z
!	"# \$% # \$	"# \$% # \$	"# \$% # \$	"# \$% # \$	"# \$% # \$
	%	%	%		
	%	%		%	
	%	%			%
		%			%



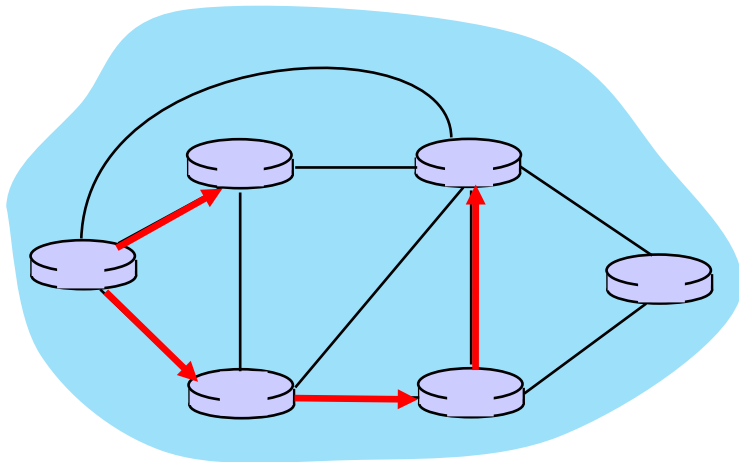
8 Loop

- 9 find a not in N' such that $D(a)$ is a minimum
- 10 add a to N'
- 11 update $D(b)$ for all b adjacent to a and not in N' :

$$D(w) = \min (D(w), D(v) + c_{v,w}) = \min (3, 2+3) = 3$$

Dijkstra's algorithm: an example

	V	W	X	Y	Z
!	"# \$% # \$	"# \$% # \$	"# \$% # \$	"# \$% # \$	"# \$% # \$
	%	%	%		
	%	%		%	
	%	%			%
		%			%

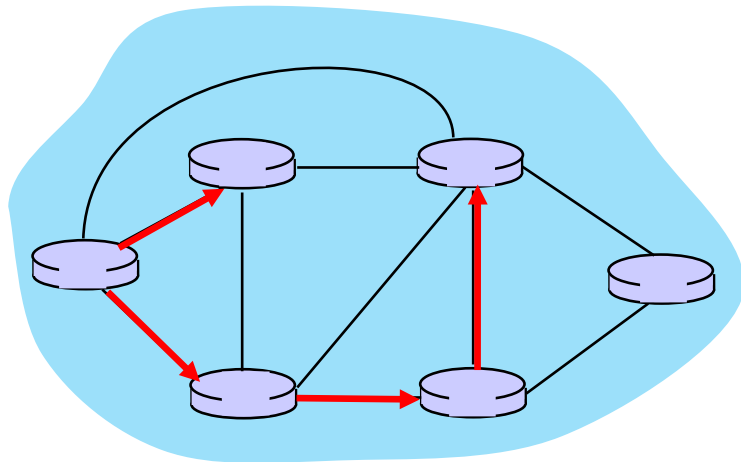


8 Loop

- 9 find a not in N' such that $D(a)$ is a minimum
- 10 add a to N'

Dijkstra's algorithm: an example

	V	W	X	Y	Z
!	"# \$% # \$	"# \$% # \$	"# \$% # \$	"# \$% # \$	"# \$% # \$
	%	%	%		
	%	%		%	
	%	%			%
		%			%
					%



8 *Loop*

9 find a not in N' such that $D(a)$ is a minimum

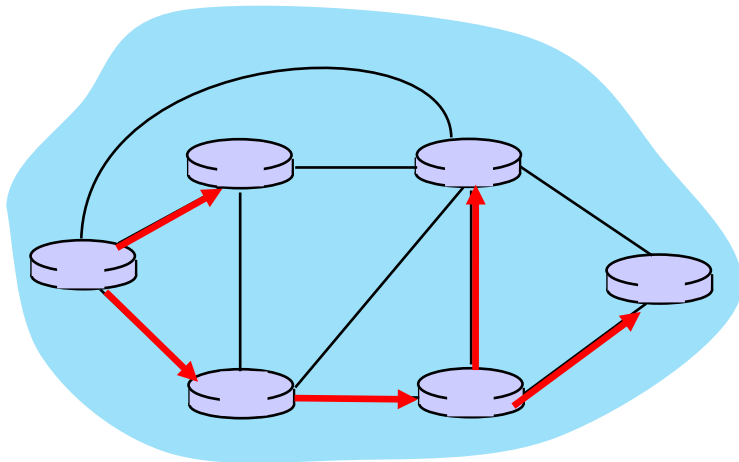
10 add a to N'

11 update $D(b)$ for all b adjacent to a and not in N' :

$$D(z) = \min (D(z), D(w) + c_{w,z}) = \min (4, 3+5) = 4$$

Dijkstra's algorithm: an example

	V	W	X	Y	Z
!	"# \$% # \$	"# \$% # \$	"# \$% # \$	"# \$% # \$	"# \$% # \$
	%	%	%		
	%	%		%	
	%	%			%
		%			%
					%

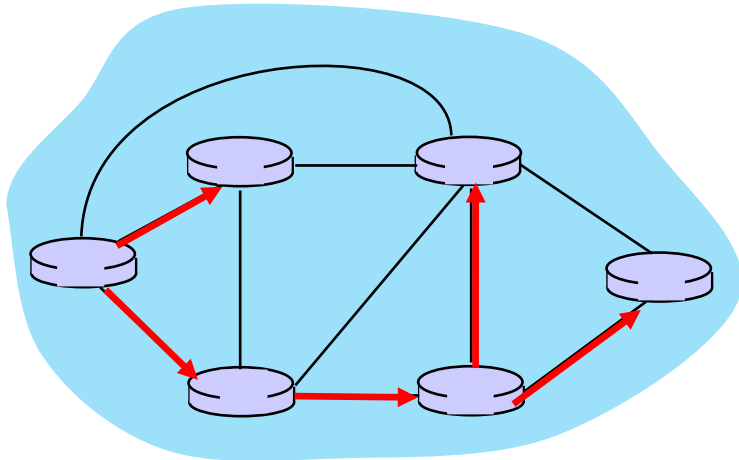


8 Loop

- 9 find a not in N' such that $D(a)$ is a minimum
- 10 add a to N'

Dijkstra's algorithm: an example

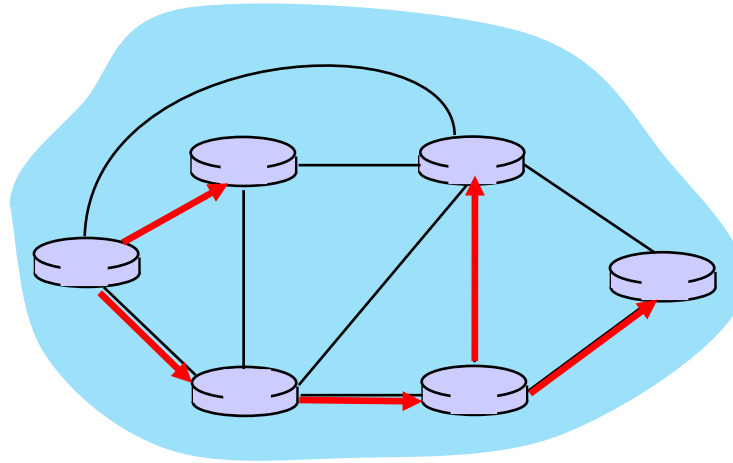
	V	W	X	Y	Z
!	"# \$% # \$	"# \$% # \$	"# \$% # \$	"# \$% # \$	"# \$% # \$
	%	%	%		
	%	%		%	
	%	%			%
		%			%
					%



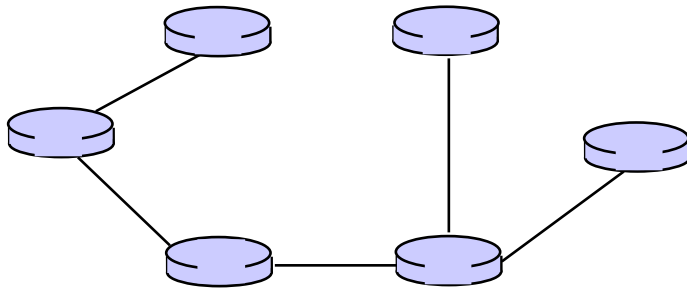
8 Loop

- 9 find a not in N' such that $D(a)$ is a minimum
- 10 add a to N'
- 11 update $D(b)$ for all b adjacent to a and not in N' :

Dijkstra's algorithm: an example



resulting least-cost-path tree from u:



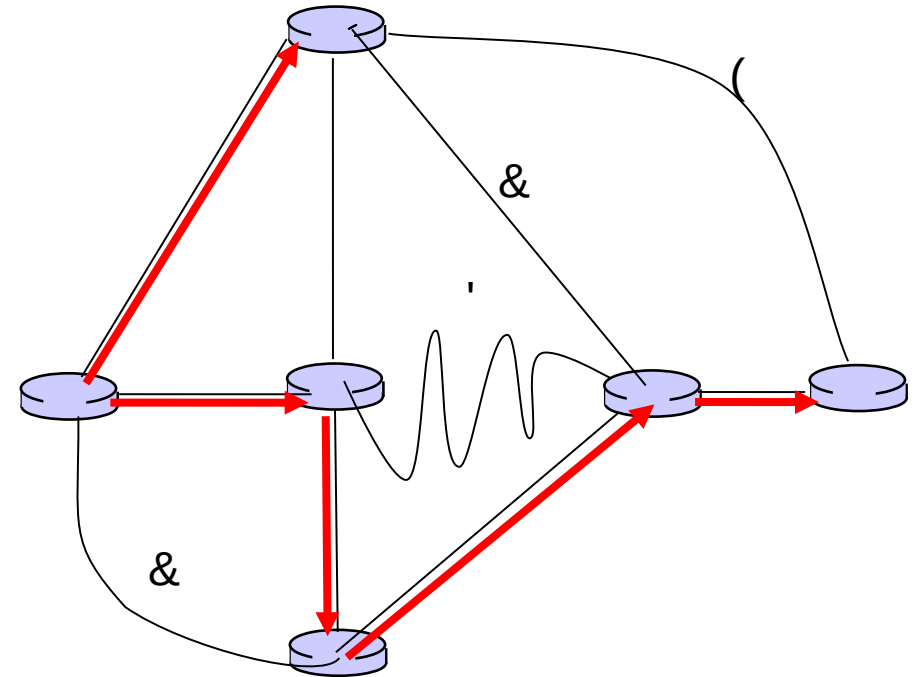
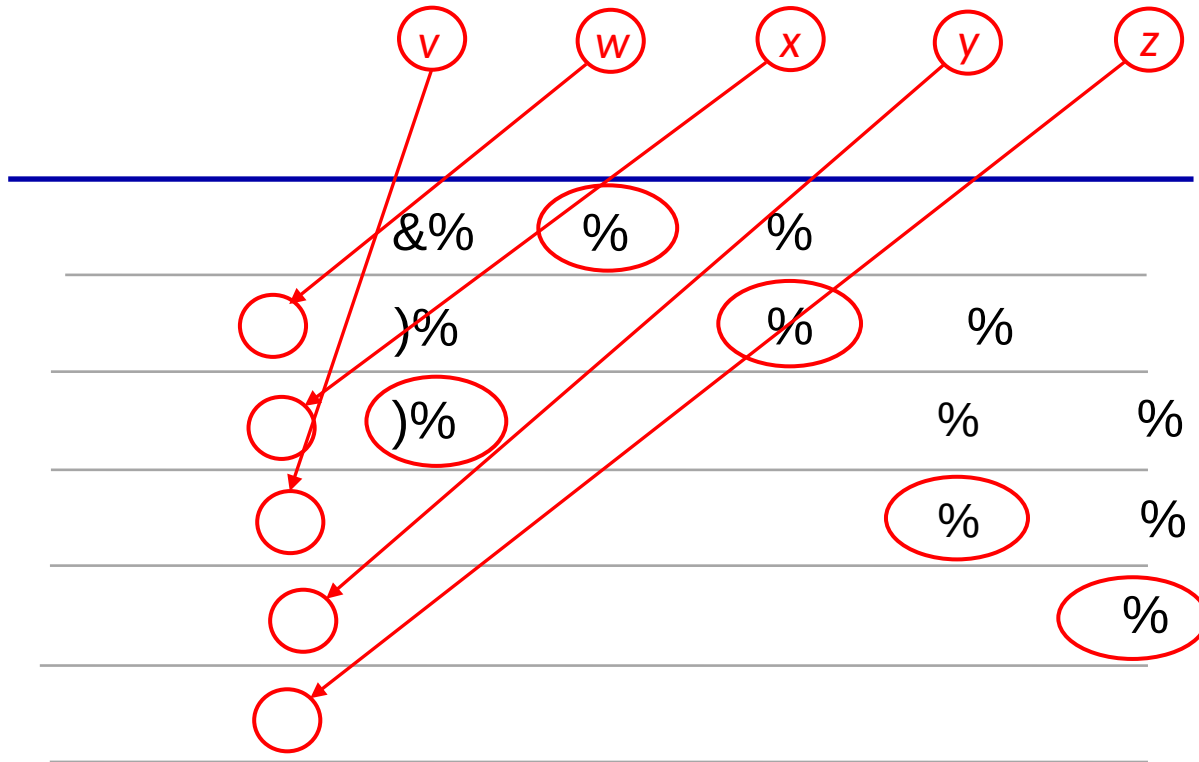
resulting forwarding table in u:

destination	outgoing link
v	(u,v)
x	(u,x)
y	(u,x)
w	(u,x)
x	(u,x)

route from u to v directly

route from u to all other destinations via x

Dijkstra's algorithm: another example



notes:

construct least-cost-path tree by tracing predecessor nodes
ties can exist (can be broken arbitrarily)

Dijkstra's algorithm: discussion

algorithm complexity: n nodes

each of n iteration: need to check all nodes, w, not in N

$n(n+1)/2$ comparisons: $O(n^2)$ complexity

more efficient implementations possible: $O(n \log n)$

message complexity:

each router must *broadcast* its link state information to other n routers

efficient (and interesting!) broadcast algorithms: $O(n)$ link crossings to disseminate a broadcast message from one source

each router's message crosses $O(n)$ links: overall message complexity: $O(n^2)$

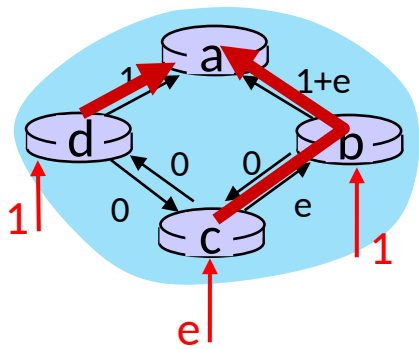
Dijkstra's algorithm: oscillations possible

when link costs depend on traffic volume, **route oscillations** possible

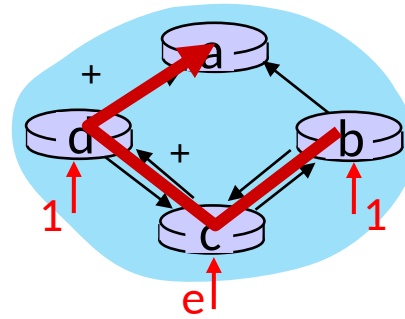
sample scenario:

routing to destination a, traffic entering at d, c, e with rates 1, e (<1), 1

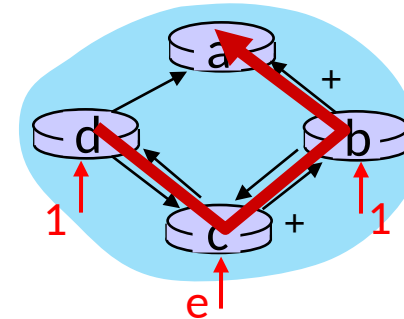
link costs are directional, and volume-dependent



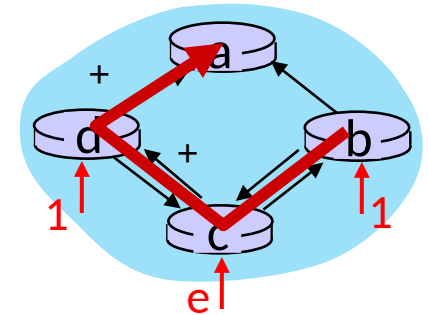
* * *



given these costs,
find new routing....
resulting in new costs



given these costs,
find new routing....
resulting in new costs



given these costs,
find new routing....
resulting in new costs

introduction

routing protocols

link state

distance vector

intra-ISP routing: OSPF

routing among ISPs: BGP

SDN control plane

Internet Control Message
Protocol



network management,
configuration

SNMP

NETCONF/YANG

Based on Bellman-Ford (BF) equation (dynamic programming):

Bellman-Ford equation

$$D_x(y) = \min_v \{ D_x(v) + c_{xv} \}$$

*

$D_x(y)$ v's estimated least-cost-path cost to y

$\min_v$ min taken over all neighbors v of x

c_{xv} direct cost of link from x to v

♦ Distance Vector (DV) Routing Algorithm – Explained

1 Big Picture: What is DV Routing?

The Distance Vector (DV) algorithm is a routing algorithm where:

- Each router only knows about its neighbors
- No router has global network knowledge
- Routers exchange information repeatedly until routes stabilize

This is very different from Link-State (LS) routing, where every router knows the full topology.

2 Key Characteristics of DV Algorithm



✓ Distributed

- No central controller
- Each router talks only to its directly connected neighbors



✓ Iterative

- Routers repeatedly exchange information
- Routes improve step by step



✓ Asynchronous

- Routers do not update at the same time
- Updates happen whenever messages arrive



✓ Self-terminating

- No explicit “stop” signal
- When no router has better information → updates naturally stop

🔴 Example protocol: RIP (Routing Information Protocol)

3 The Bellman-Ford Equation (Core Idea)

The DV algorithm is based on the Bellman-Ford equation:

$$d_x(y) = \min_v \{c(x, v) + d_v(y)\}$$

Meaning:

The shortest path from $x \rightarrow y$ is:

Go from $x \rightarrow$ some neighbor v , then take v 's best path to y

🔍 Why this makes sense

- You must leave node x via one of its neighbors
- So you check all neighbors
- Pick the neighbor that gives the lowest total cost

4 How Forwarding Tables Come from Bellman-Ford

If neighbor v^* gives the minimum cost:

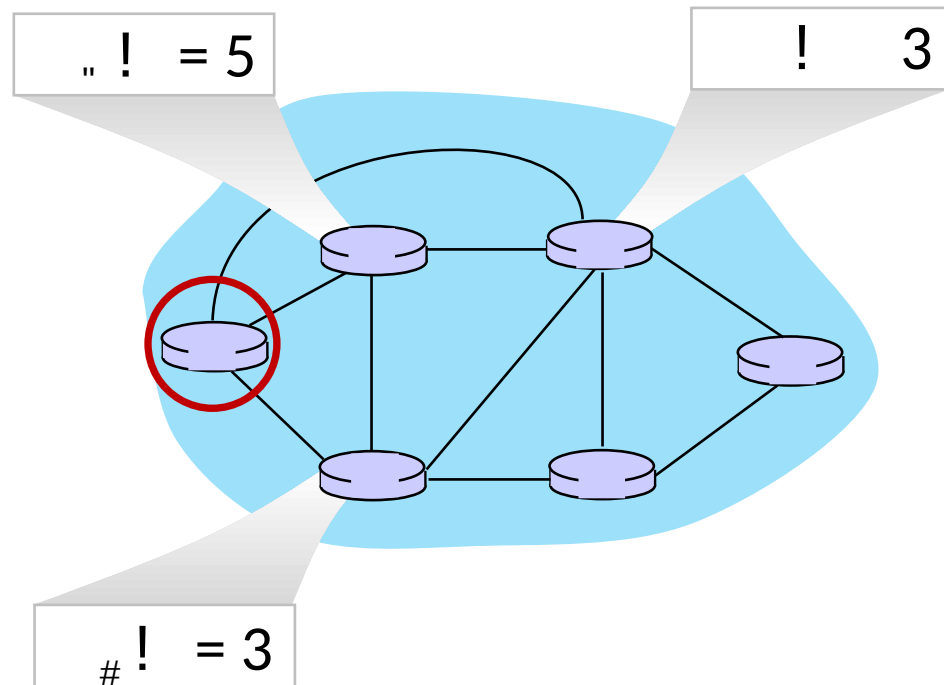
$$v^* = \arg \min_v \{c(x, v) + d_v(y)\}$$

Then:

- Next hop to destination $y = v^*$
- This is exactly how routing tables are built

Bellman-Ford Example

Suppose that node u 's neighboring node v knows that for destination w :



Bellman-Ford equation says:

$$\begin{aligned}
 & \text{Distance}(u, w) = \min_v \{ \text{Distance}(u, v) + \text{Distance}(v, w) \} \\
 & \text{Distance}(u, w) = \min \{ 1 + 2, 3 + 1 \} \\
 & \text{Distance}(u, w) = 3
 \end{aligned}$$

Distance vector algorithm RIP uses this

key idea:

- from time-to-time, each node sends its own distance vector estimate to neighbors
- when # receives new DV estimate from any neighbor, it updates its own DV using B-F equation:

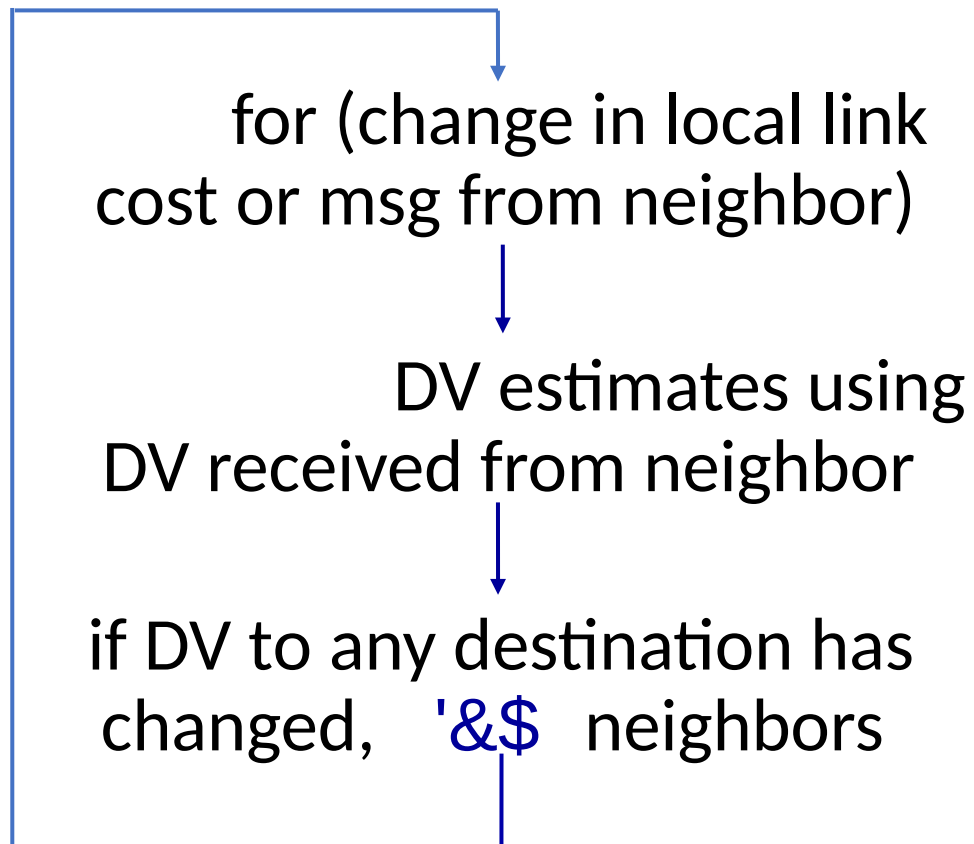
$$\# \$ 7 \quad \text{"8 #"} / \quad \text{" \$ 9 for each node \$}$$

- under minor, natural conditions, the estimate $\# \$$ "

$$\min_y \{ \text{"} \% \% * * d_x(y) \}$$

Distance vector algorithm:

each node:



iterative, asynchronous: each local iteration caused by:

- local link cost change
- DV update message from neighbor

distributed, self-stopping: each node notifies neighbors %\$ when its DV changes

- neighbors then notify their neighbors - %\$ & ** \$
- no notification received, no actions taken!

Distance vector: example

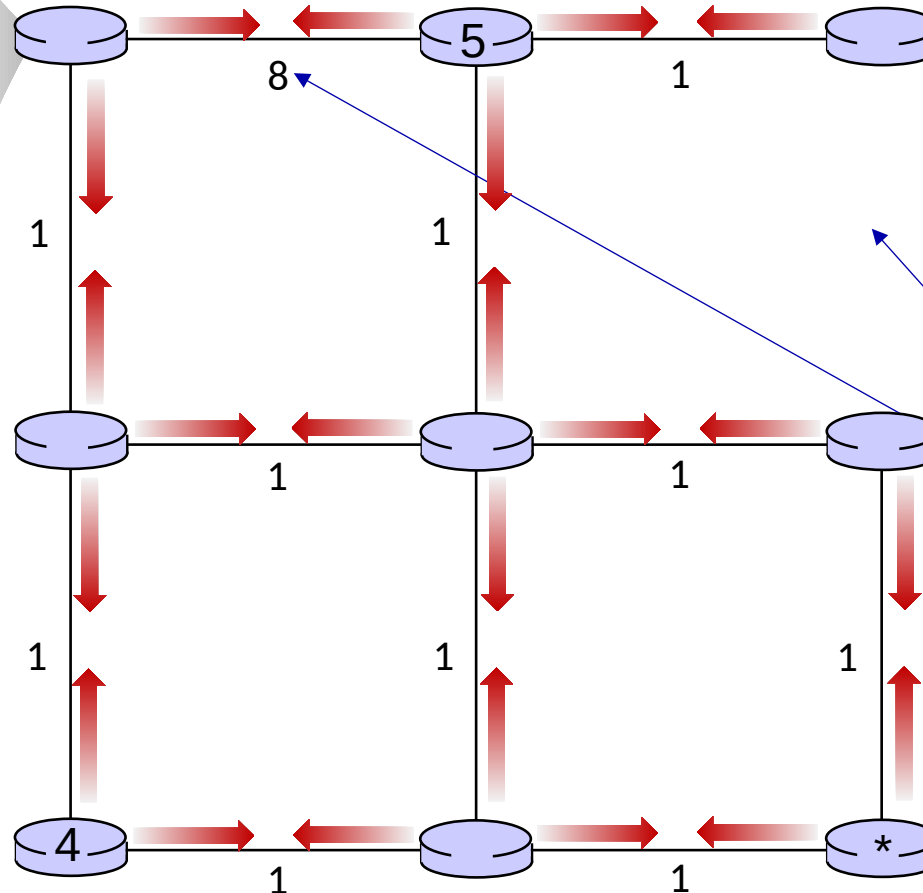


t=0

All nodes have distance estimates to nearest neighbors (only)

All nodes send their local distance vector to their neighbors

DV in a:
$D_a(a)=0$
$D_a(b)=8$
$D_a(c)=\infty$
$D_a(d)=1$
$D_a(e)=\infty$
$D_a(f)=\infty$
$D_a(g)=\infty$
$D_a(h)=\infty$
$D_a(i)=\infty$



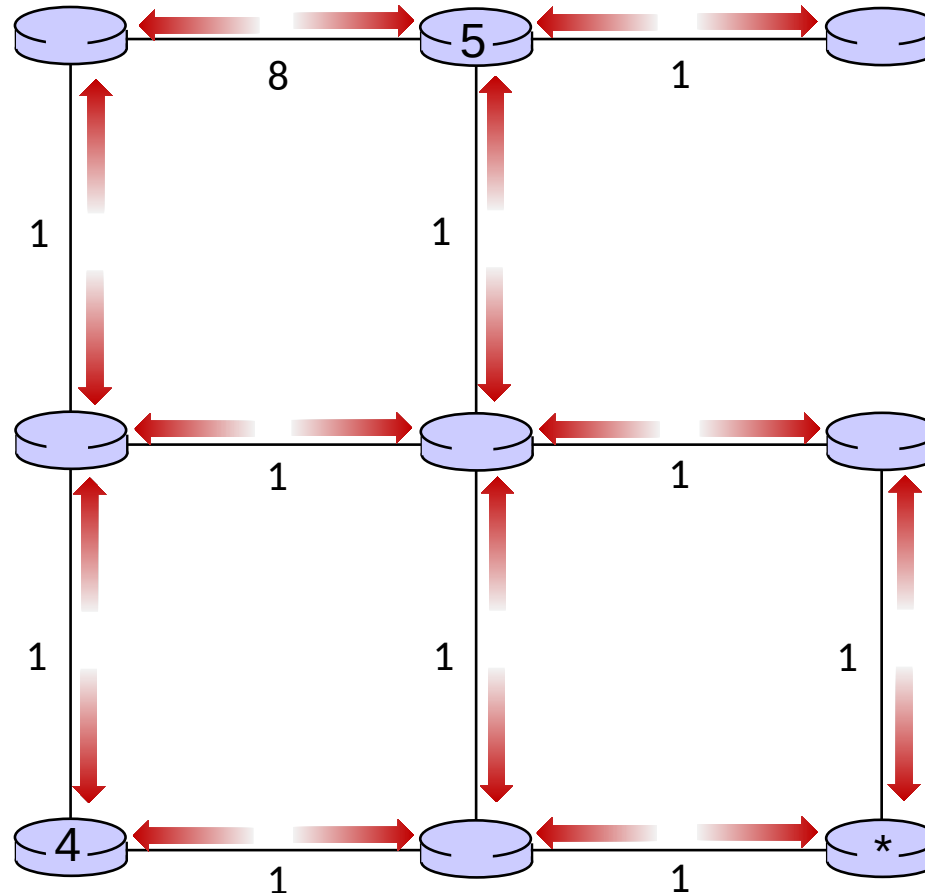
A few asymmetries:
missing link
larger cost

Distance vector example: iteration



$t=1$

All nodes:
receive distance
vectors from
neighbors
compute their new
local distance
vector
send their new
local distance
vector to neighbors



Distance vector example: iteration



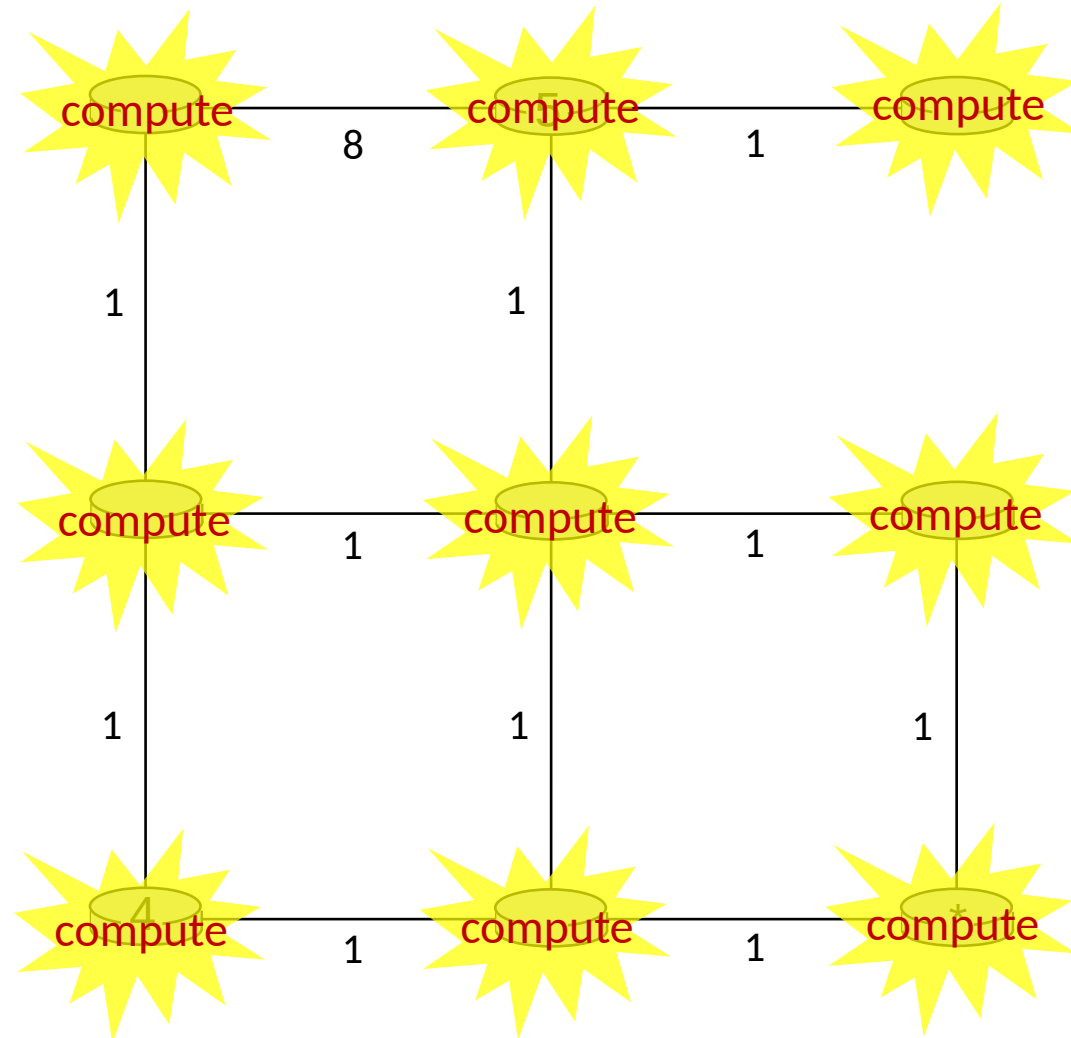
$t=1$

All nodes:

receive distance
vectors from
neighbors

compute their new
local distance
vector

send their new
local distance
vector to neighbors



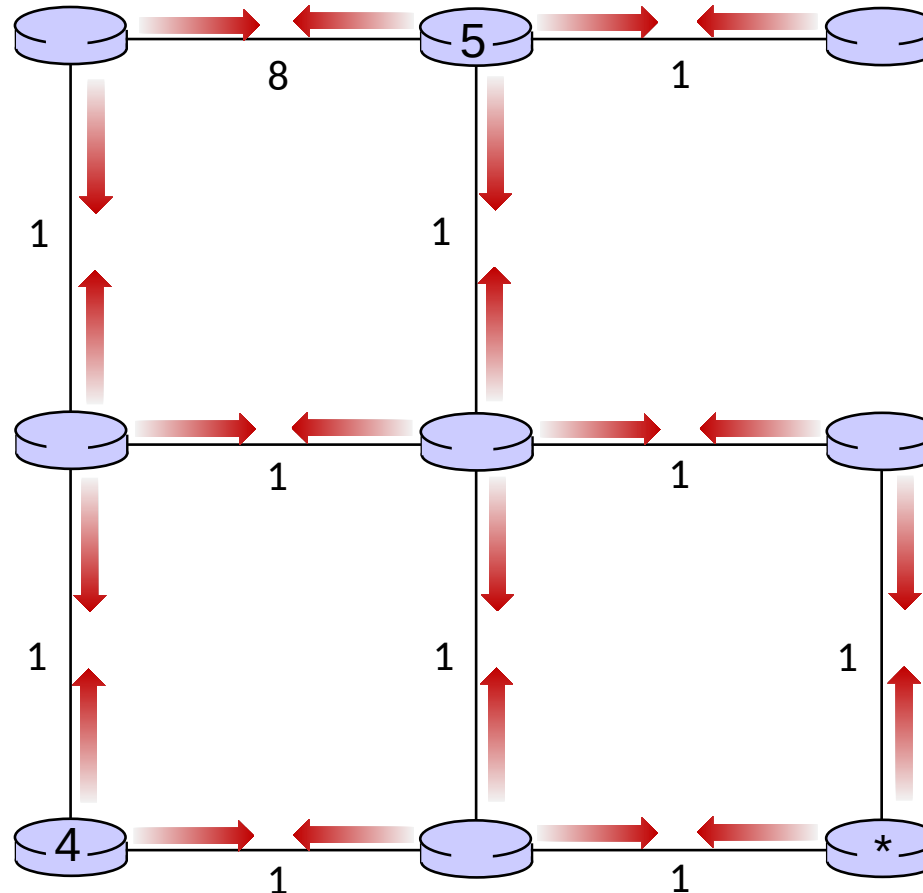
Distance vector example: iteration



$t=1$

All nodes:

receive distance
vectors from
neighbors
compute their new
local distance
vector
send their new
local distance
vector to neighbors

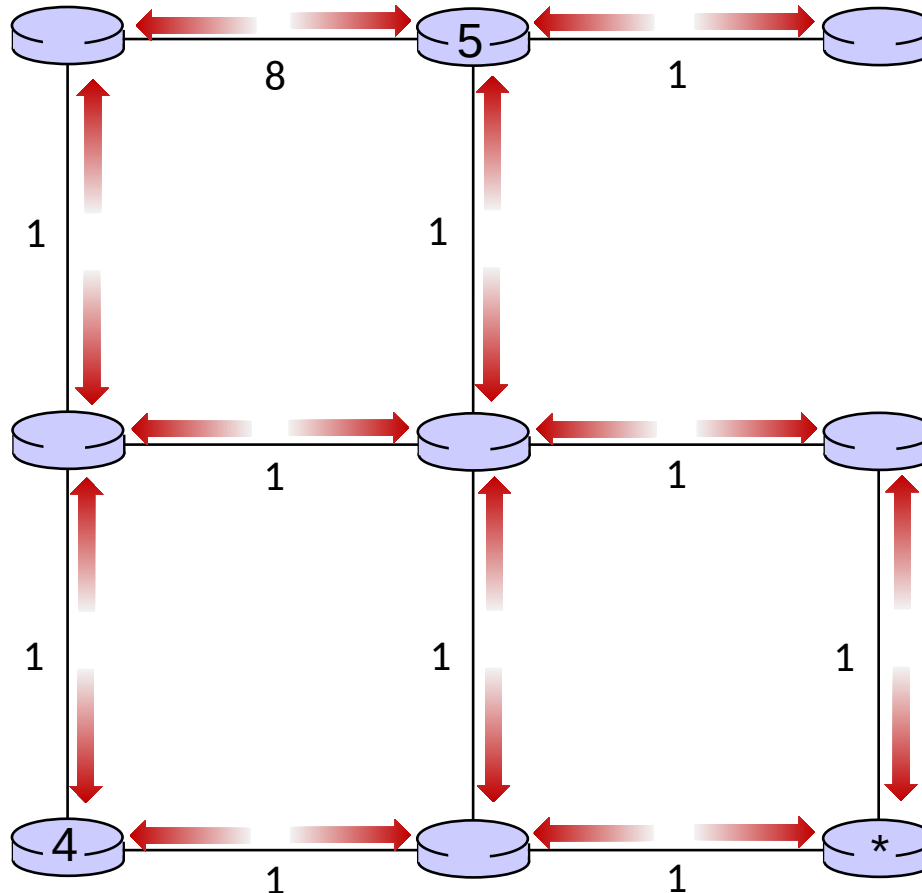


Distance vector example: iteration



$t=2$

All nodes:
receive distance
vectors from
neighbors
compute their new
local distance
vector
send their new
local distance
vector to neighbors



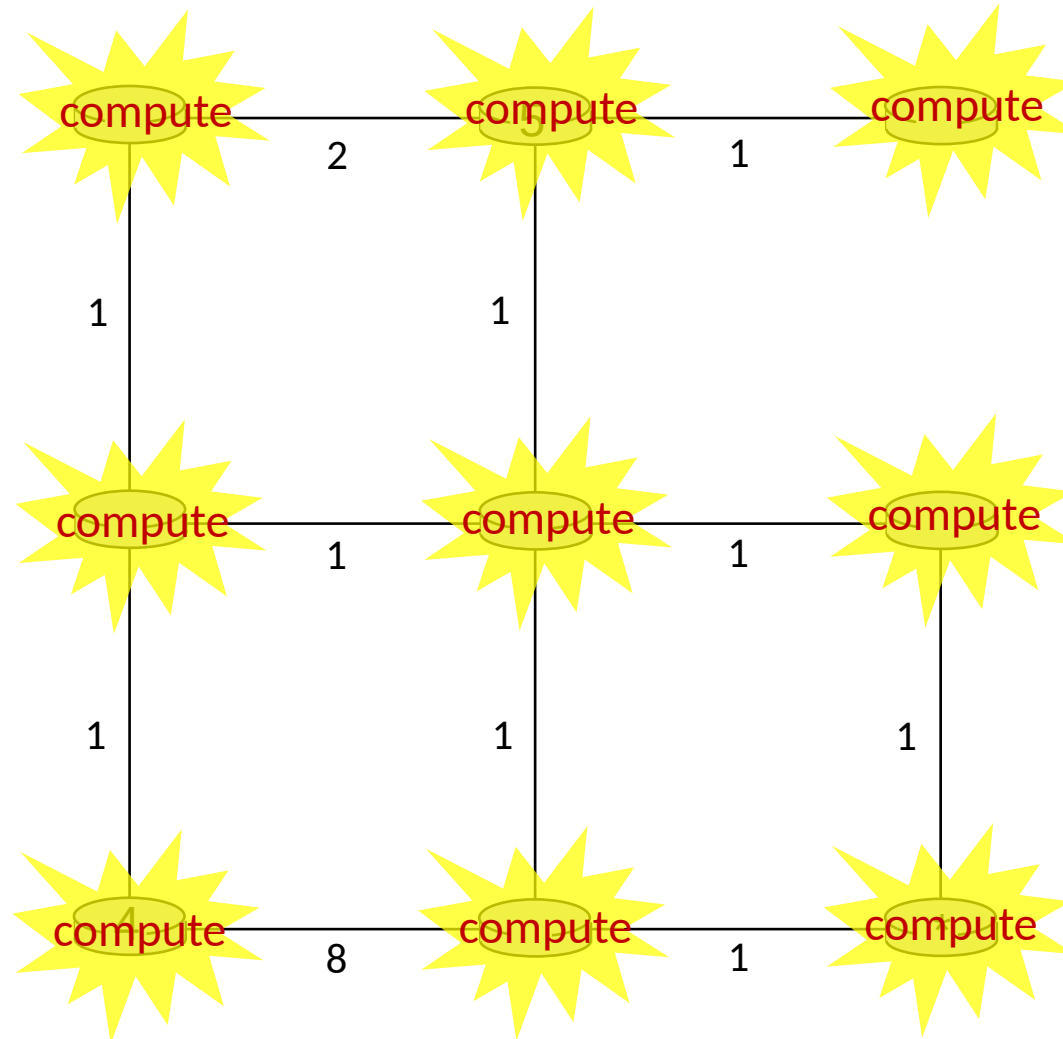
Distance vector example: iteration



t=2

All nodes:

receive distance
vectors from
neighbors
compute their new
local distance
vector
send their new
local distance
vector to neighbors



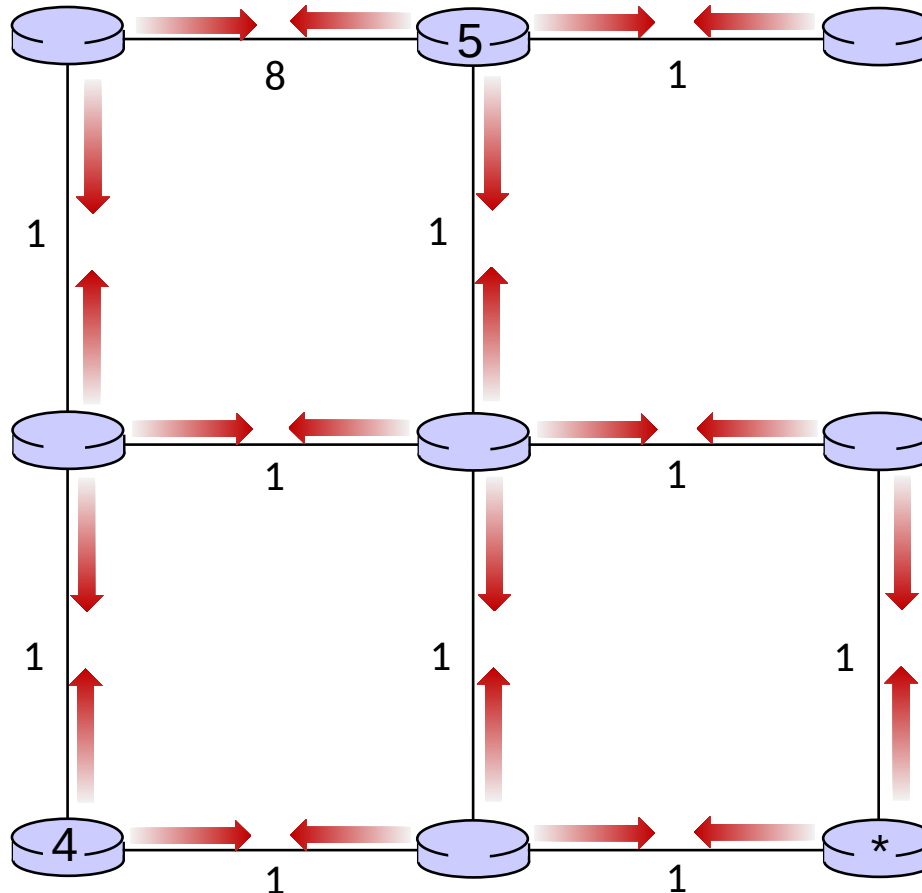
Distance vector example: iteration



$t=2$

All nodes:

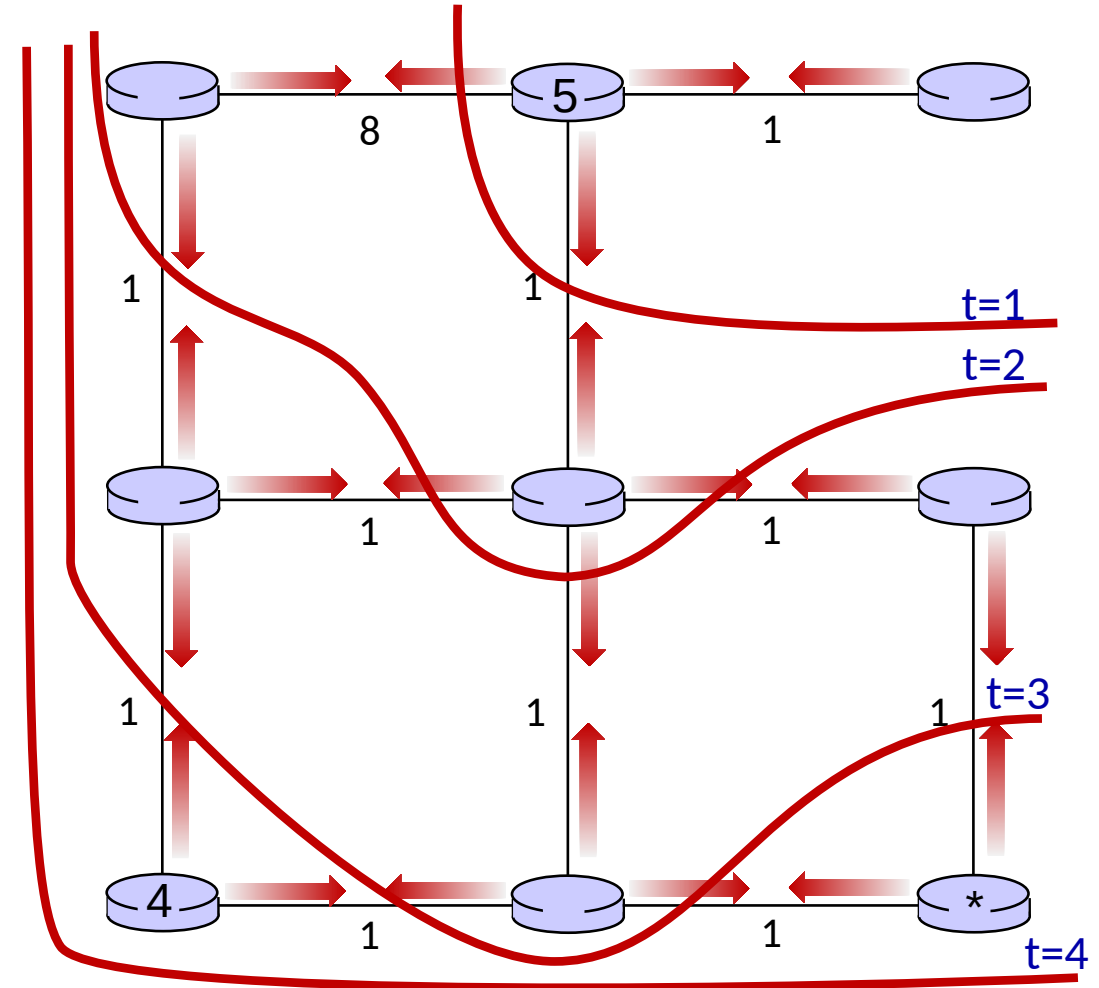
receive distance
vectors from
neighbors
compute their new
local distance
vector
send their new
local distance
vector to neighbors



Distance vector: state information diffusion

Iterative communication, computation steps diffuses information through network:

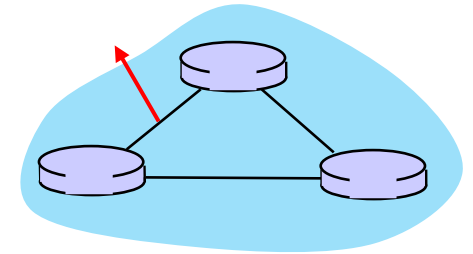
-  $t=0$ c's state at $t=0$ is at c only
-  $t=1$ c's state at $t=0$ has propagated to b, and may influence distance vector computations up to 1 hop away, i.e., at b
-  $t=2$ c's state at $t=0$ may now influence distance vector computations up to 2 hops away, i.e., at b and now at a, e as well
-  $t=3$ c's state at $t=0$ may influence distance vector computations up to 3 hops away, i.e., at d, f, h
-  $t=4$ c's state at $t=0$ may influence distance vector computations up to 4 hops away, i.e., at g, i



Distance vector: link cost changes

link cost changes:

node detects local link cost change
updates routing info, recalculates local DV
if DV changes, notify neighbors



“good news
travels fast”

0 : \$ detects link-cost change, updates its DV, informs its neighbors.

1 : ! receives update from \$, updates its DV, computes new least cost to #, sends its neighbors its DV.

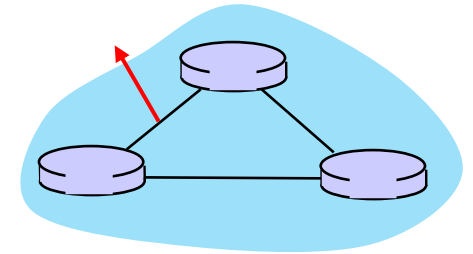
0 : \$ receives !'s update, updates its DV. \$'s least costs do change, so \$ does send a message to !.

Distance vector: link cost changes

link cost changes:

node detects local link cost change

“bad news travels slow” – count-to-infinity problem:



\$ sees direct link to # has new cost 60, but z has said it has a path at cost of 5. So \$ computes “my new cost to x will be 6, via z); notifies ! of new cost of 6 to #

! learns that path to # via \$ has new cost 6, so ! computes “my new cost to # will be 7 via y), notifies \$ of new cost of 7 to #

\$ learns that path to # via ! has new cost 7, so \$ computes “my new cost to # will be 8 via y), notifies ! of new cost of 8 to #

! learns that path to # via \$ has new cost 8, so ! computes “my new cost to # will be 9 via y), notifies \$ of new cost of 9 to #

...

see text for solutions. * % * \$<

Comparison of LS and DV algorithms

message complexity

LS: routers, $O(N^2)$ messages sent

DV: exchange between neighbors;
convergence time varies

speed of convergence

LS: $O(N^2)$ algorithm, $O(N^2)$ messages
may have oscillations

DV: convergence time varies
may have routing loops
count-to-infinity problem

robustness: what happens if router malfunctions, or is compromised?

LS:

router can advertise incorrect cost
each router computes only its table

DV:

DV router can advertise incorrect cost ("I have a low-cost path to everywhere"): ∞ ∞

each router's DV is used by others:
error propagate thru network



Distance Vector Routing	Link State Routing
Bandwidth required is less due to local sharing, small packets and no flooding.	Bandwidth required is more due to flooding and sending of large link state packets.
Based on local knowledge, since it updates table based on information from neighbours.	Based on global knowledge, it have knowledge about entire network.
Make use of Bellman Ford Algorithm .	Make use of Dijkstra's algorithm .
Traffic is less.	Traffic is more.
Converges slowly i.e, good news spread fast and bad news spread slowly.	Converges faster.
Count of infinity problem .	No count of infinity problem.
Persistent looping problem i.e, loop will be there forever.	No persistent loops, only transient loops.
Practical implementation is RIP and IGRP .	Practical implementation is OSPF and ISIS .



Making routing scalable

our routing study thus far - idealized

- all routers identical
- network “flat”

... not true in practice

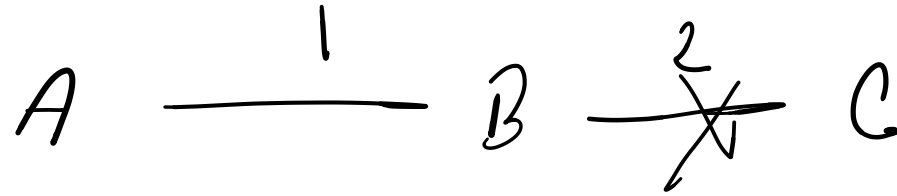
scale: billions of destinations:

- can't store all destinations in routing tables!
- routing table exchange would swamp links!

administrative autonomy:

- Internet: a network of networks
- each network admin may want to control routing in its own network

Count to infinity Problem



A

T	d	Via
B	1	B
C	3	B

5
7

B → A

A → B

B

T	d	Via
A	1	B
C	2	B

C → A

C

A → C

C

T	d	Via
A	3	B
B	2	B

Intra-Domain vs Inter-Domain Routing

♦ Basic Idea

- Intra-domain routing → routing within a single Autonomous System (AS)
- Inter-domain routing → routing between different Autonomous Systems



Tabular Comparison

Feature	Intra-Domain Routing	Inter-Domain Routing 
Scope	Within one AS	Between multiple ASes
Controlled by	Single organization	Multiple independent organizations
Routing goal	Shortest / least-cost path 	Policy-based routing 
Routing protocol type	IGP (Interior Gateway Protocol)	EGP (Exterior Gateway Protocol) 
Common protocols	RIP, OSPF, IS-IS	BGP 
Metric used	Hop count, cost, bandwidth, delay 	Path attributes & policies 
Routing decisions	Performance-oriented	Business & policy-oriented
Trust level	High (same admin control)	Low (between organizations)
Convergence speed	Fast	Slow (for stability)
Routing updates	Frequent	Less frequent
Scalability	Limited to one AS	Internet-scale
Loop prevention	Metrics, sequence numbers	AS-Path
Example	University campus network	ISP-to-ISP routing

Intra-Domain Routing (IGP)

✦ Definition

Routing inside a single Autonomous System under one administrative authority.

✦ Characteristics

- Optimizes network performance
- Uses numerical metrics
- Rapid convergence after failures
- Simpler trust assumptions

✦ Examples

- RIP → Distance Vector, hop count
- OSPF → Link State, cost
- IS-IS → Link State, large networks

📌 Used by:

- Enterprise networks
- Campus networks
- Data centers

Inter-Domain Routing (EGP)

✦ Definition

Routing between different Autonomous Systems across the Internet.

✦ Characteristics

- Policy-driven, not shortest path
- Routing depends on:
 - Business relationships
 - Traffic agreements
 - Security rules
- Slower convergence to avoid global instability

✦ Protocol Used

- BGP (Border Gateway Protocol)

✦ BGP Decision Factors

- AS-Path length
- Local preference
- MED (Multi-Exit Discriminator)
- Policy rules

📌 Used by:

- ISPs
- Internet backbone routers 

+

aggregate routers into regions known as “autonomous systems” (AS) (a.k.a. “domains”)

intra-AS (aka “intra-domain”):
routing among routers *

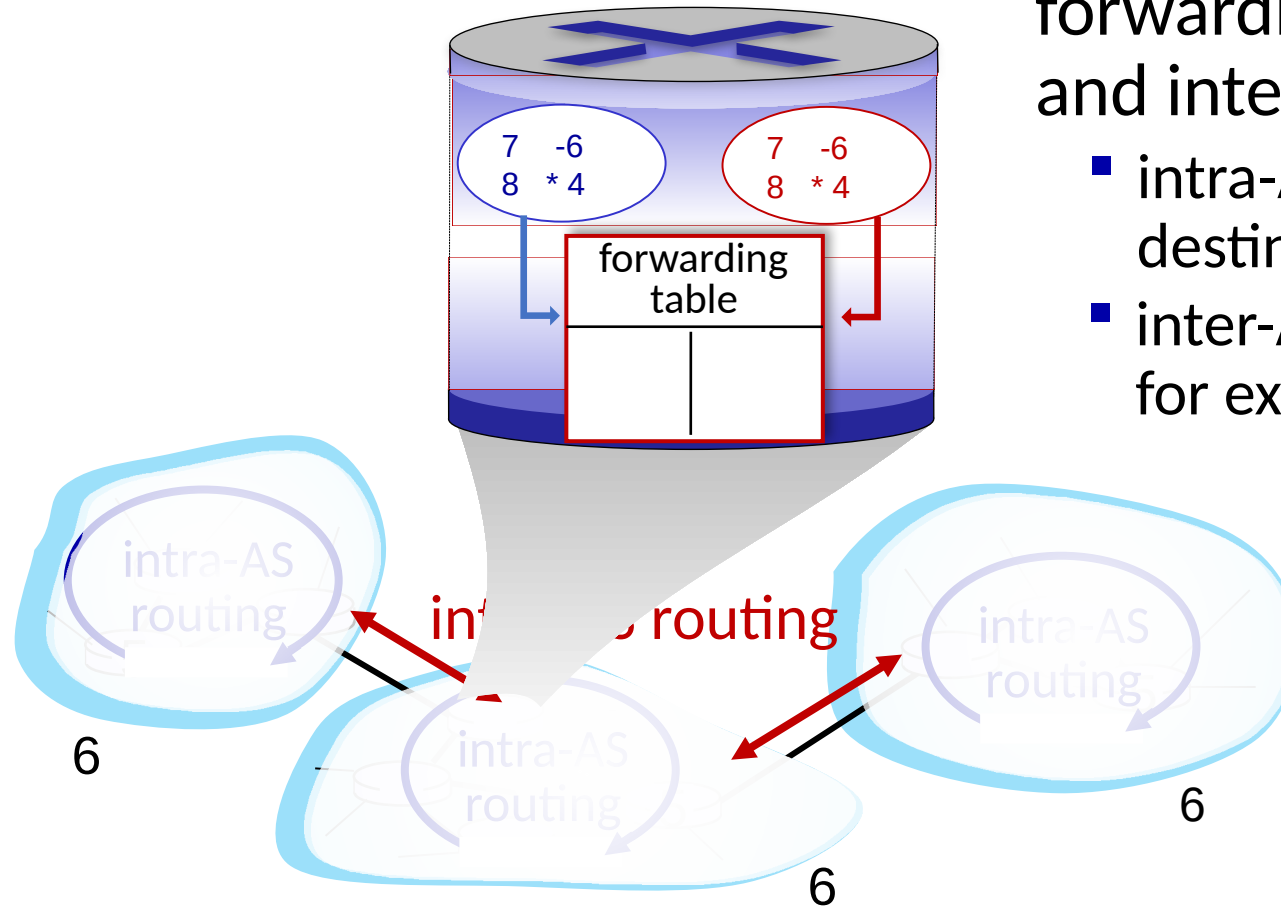
= > ?

- all routers in AS must run same intra-domain protocol
- routers in different AS can run different intra-domain routing protocols
- **gateway router**: at “edge” of its own AS, has link(s) to router(s) in other AS'es

inter-AS (aka “inter-domain”):
routing AS'es

- gateways perform inter-domain routing (as well as intra-domain routing)

Interconnected ASes



forwarding table configured by intra- and inter-AS routing algorithms

- intra-AS routing determine entries for destinations within AS
- inter-AS & intra-AS determine entries for external destinations

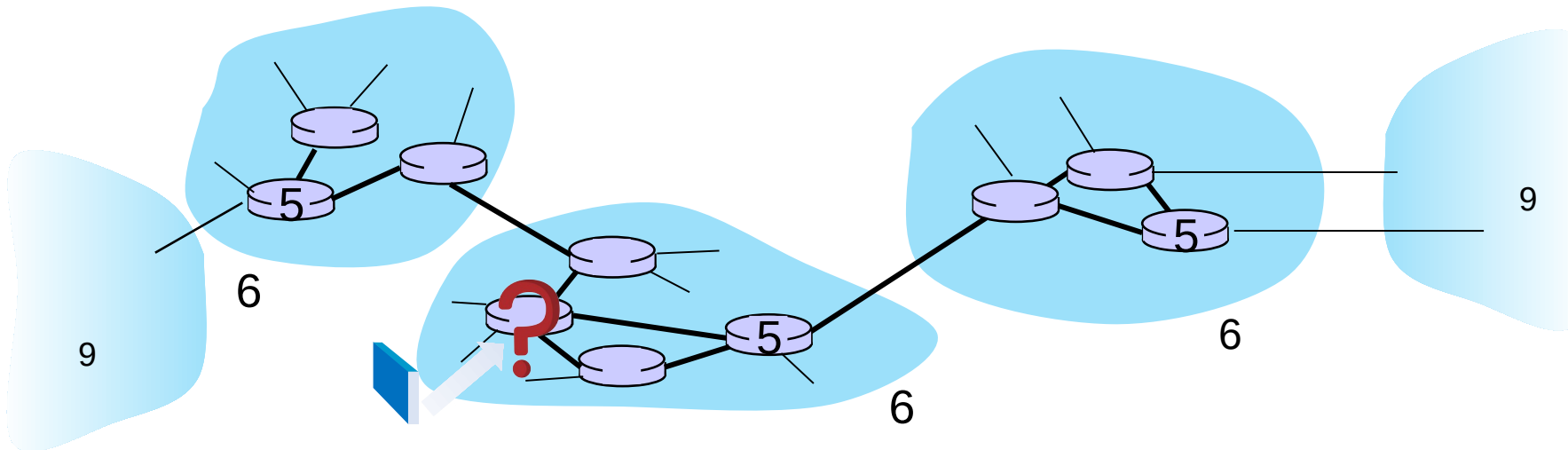
Inter-AS routing: a role in intradomain forwarding

- suppose router in AS receives datagram destined outside of AS :

? router should forward packet to gateway router in AS1, but which one?

AS1 inter-domain routing must:

- learn which destinations reachable through AS2, which through AS3
- propagate this reachability info to all routers in AS1



Intra-AS routing: routing within an AS

most common intra-AS routing protocols:

- **RIP: Routing Information Protocol** [RFC 1723]

classic DV: DVs exchanged every 30 secs
no longer widely used

- **EIGRP: Enhanced Interior Gateway Routing Protocol**

DV based
formerly Cisco-proprietary for decades (became open in 2013 [RFC 7868])

- **OSPF: Open Shortest Path First** [RFC 2328]

link-state routing
IS-IS protocol (ISO standard, not RFC standard) essentially same as OSPF

,) # -,) # .

- “open”: publicly available
- classic link-state
 - each router floods OSPF link-state advertisements (directly over IP rather than using TCP/UDP) to all other routers in entire AS
 - multiple link costs metrics possible: bandwidth, delay
 - each router has full topology, uses Dijkstra’s algorithm to compute forwarding table
- * \$ all OSPF messages authenticated (to prevent malicious intrusion)

/ ,) #

- **two-level hierarchy:** local area, backbone.

link-state advertisements flooded only in area, or backbone

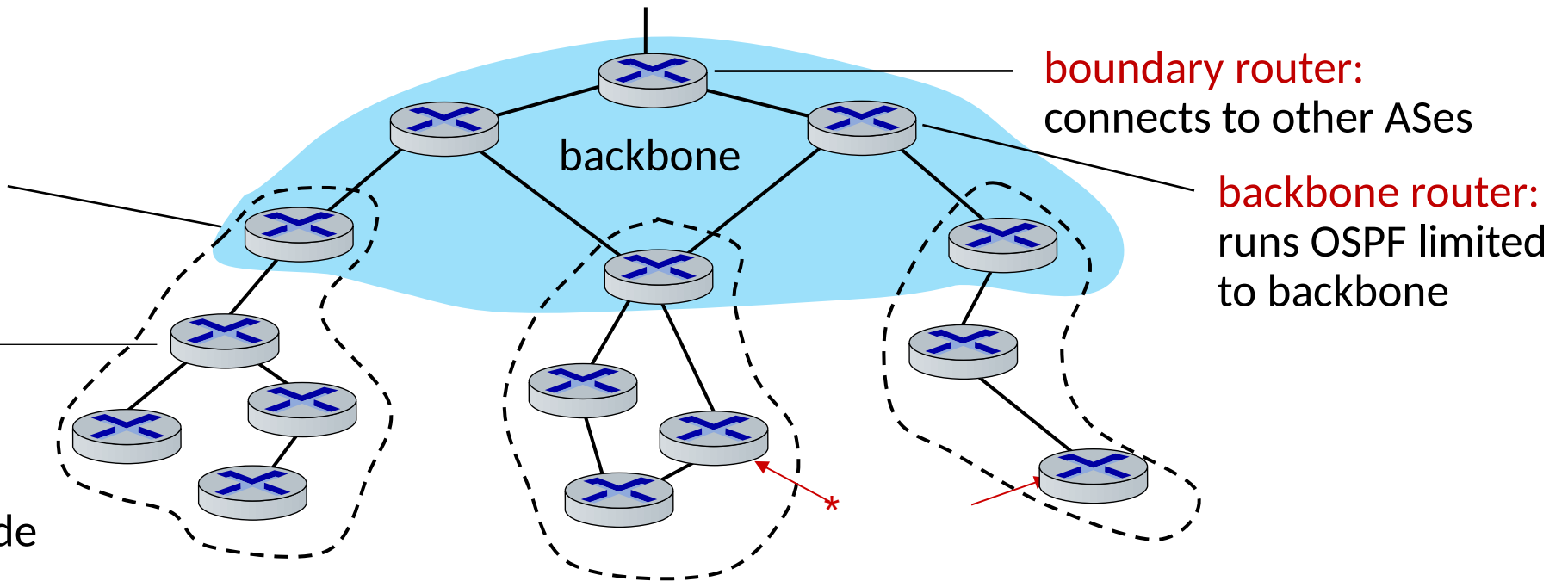
each node has detailed area topology; only knows direction to reach other destinations

area border routers:

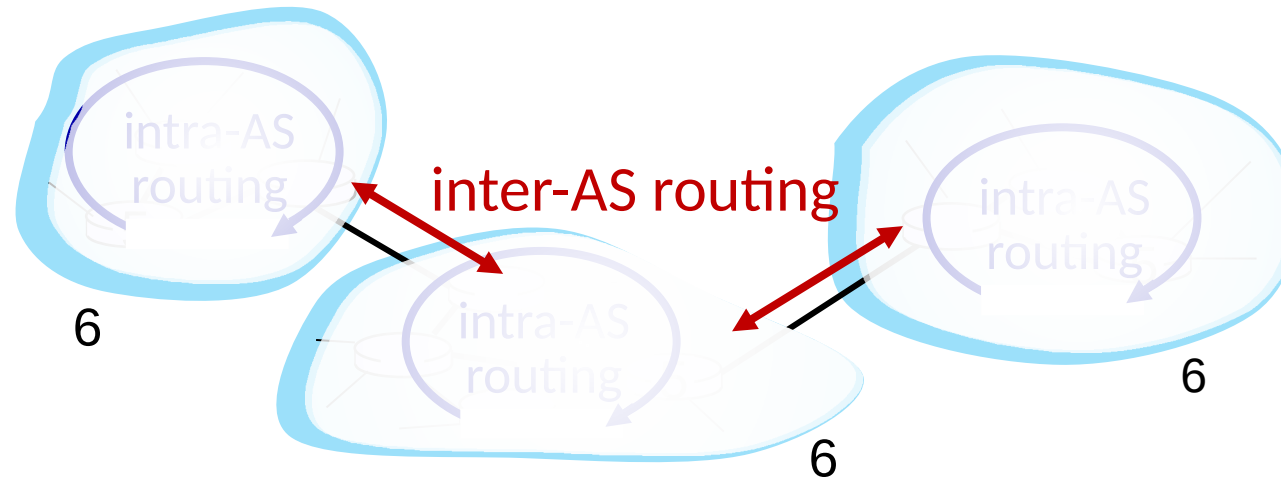
“summarize” distances to destinations in own area, advertise in backbone

local routers:

flood LS in area only
compute routing within area
forward packets to outside via area border router



Interconnected ASes



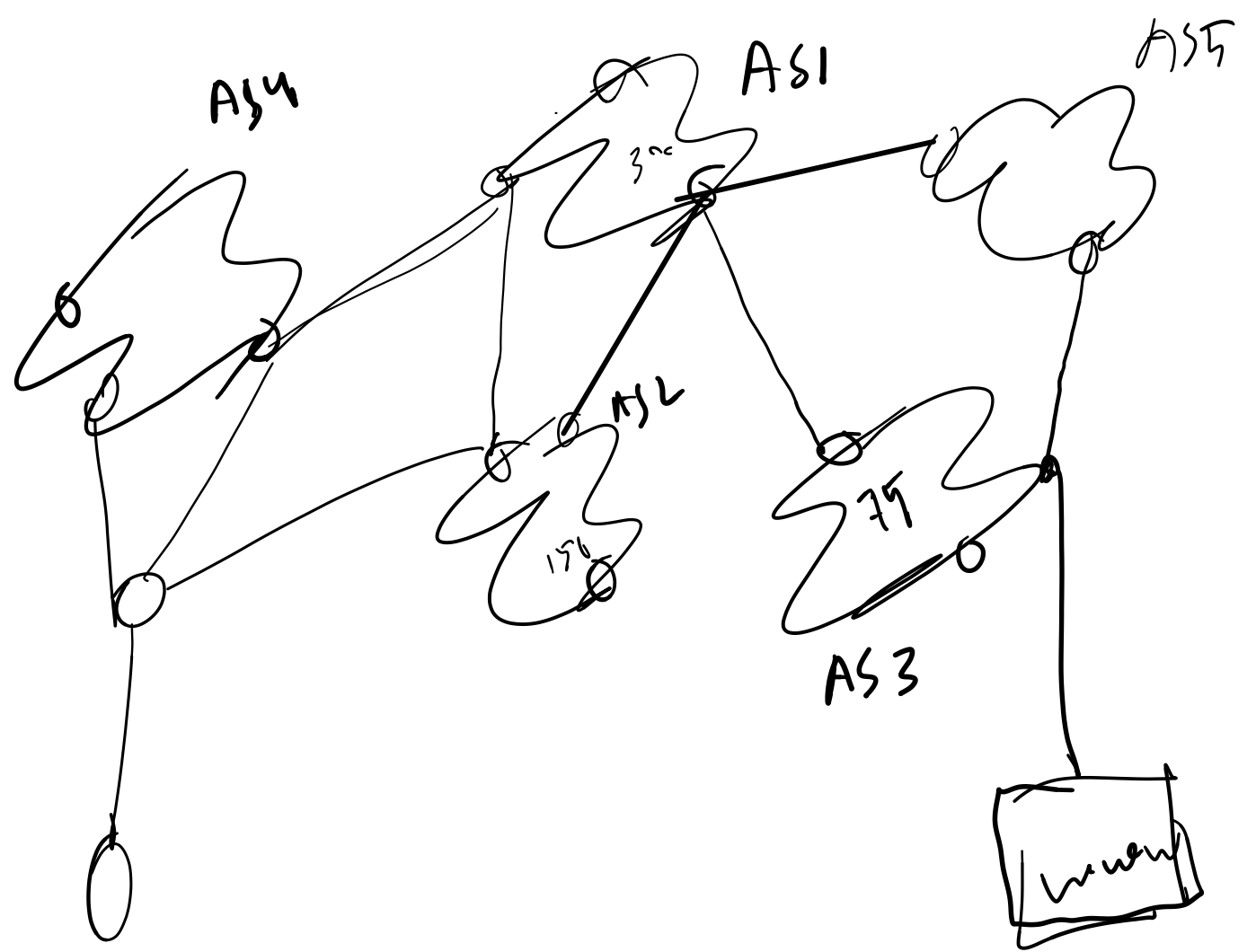
intra-AS (aka “intra-domain”): routing among routers

*

= > ?



inter-AS (aka “inter-domain”): routing AS'es

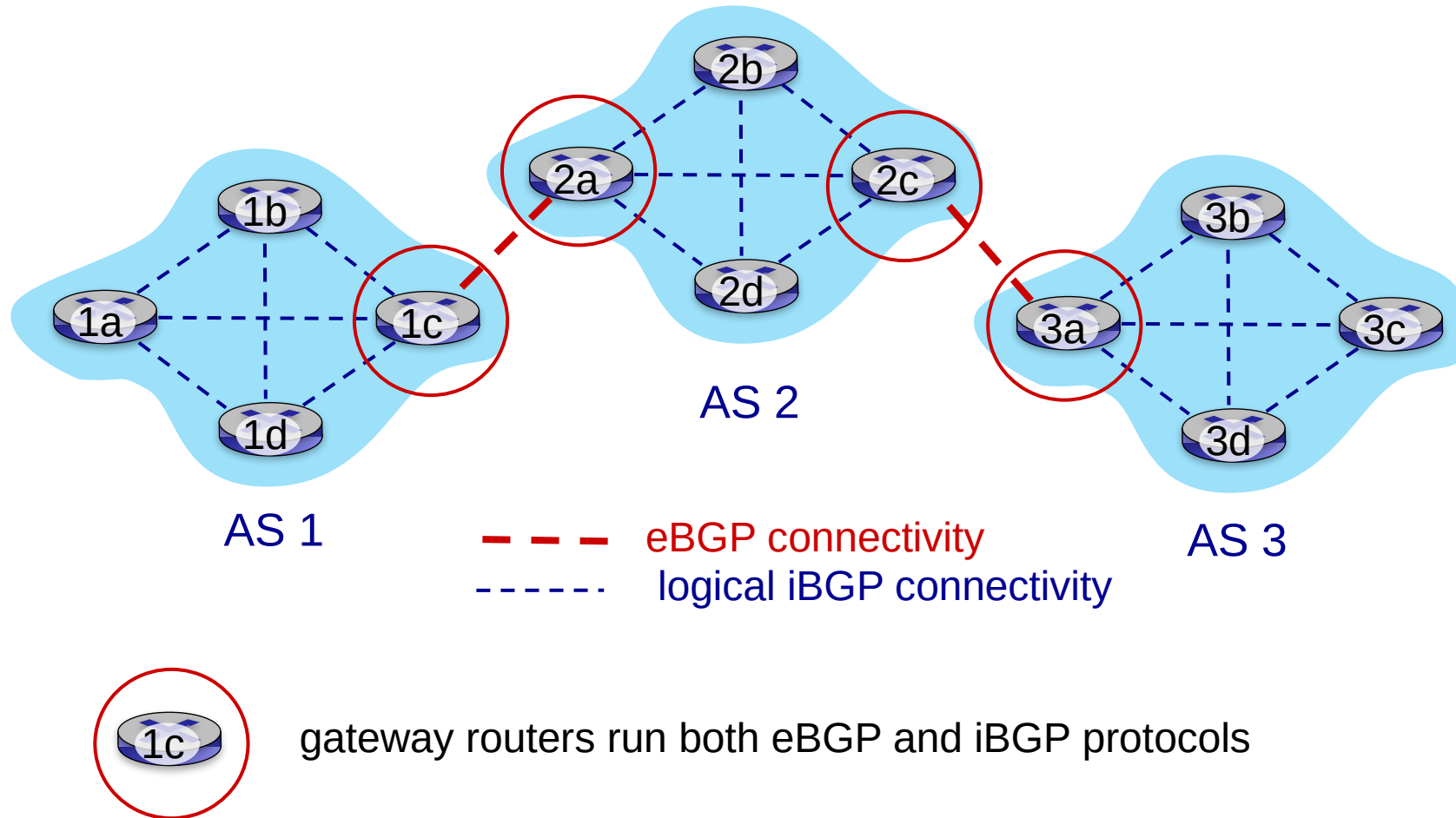


125.4.5.0/24

Internet inter-AS routing: BGP

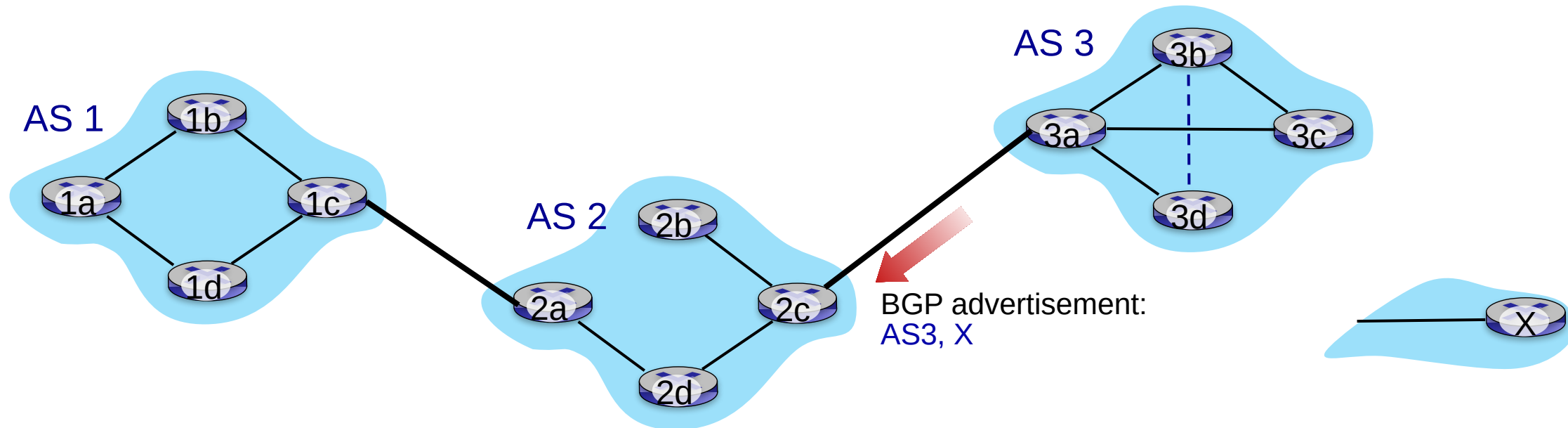
- BGP (Border Gateway Protocol): de facto inter-domain routing protocol
 - “glue that holds the Internet together”
- allows subnet to advertise its existence, and the destinations it can reach, to rest of Internet: >- * - ?
- BGP provides each AS a means to:
 - obtain destination network reachability info from neighboring ASes (eBGP)
 - determine routes to other networks based on reachability information and % \$
 - propagate reachability information to all AS-internal routers (iBGP)
 - **advertise** (to neighboring networks) destination reachability info

eBGP, iBGP connections



BGP basics

- **BGP session:** two BGP routers (“peers”) exchange BGP messages over semi-permanent TCP connection:
 - advertising * to different destination network prefixes (BGP is a “path vector” protocol)
- when AS3 gateway 3a advertises **path AS3,X** to AS2 gateway 2c:
 - AS3 * * to AS2 it will forward datagrams towards X



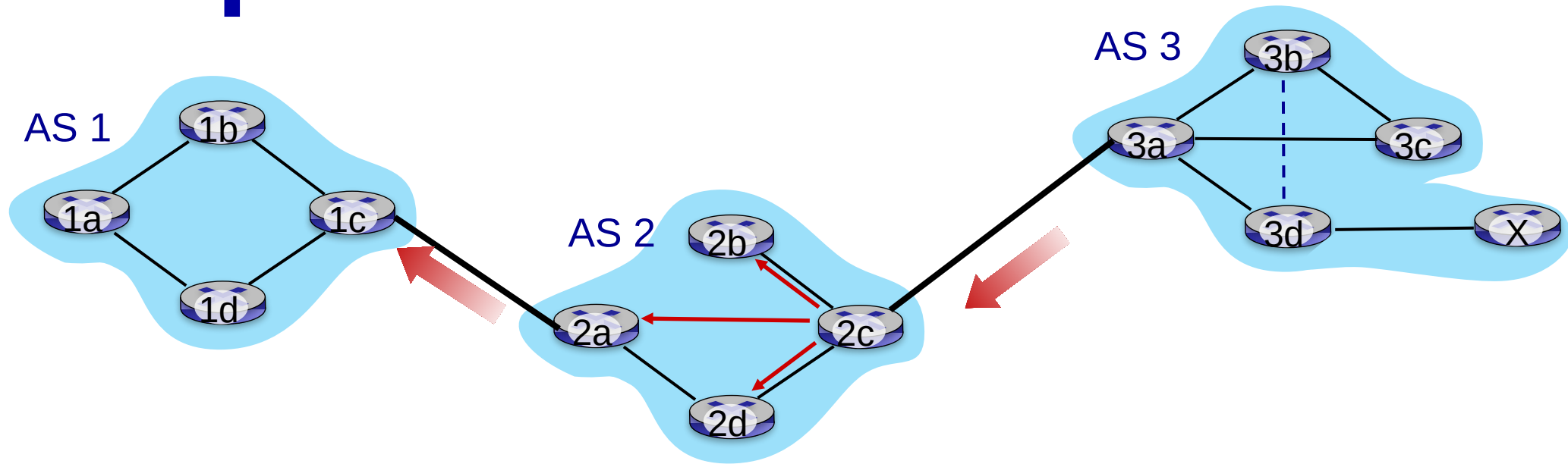
BGP protocol messages

- BGP messages exchanged between peers over TCP connection
- BGP messages [RFC 4371]:
 - **OPEN**: opens TCP connection to remote BGP peer and authenticates sending BGP peer
 - **UPDATE**: advertises new path (or withdraws old)
 - **KEEPALIVE**: keeps connection alive in absence of UPDATES; also ACKs OPEN request
 - **NOTIFICATION**: reports errors in previous msg; also used to close connection

Path attributes and BGP routes

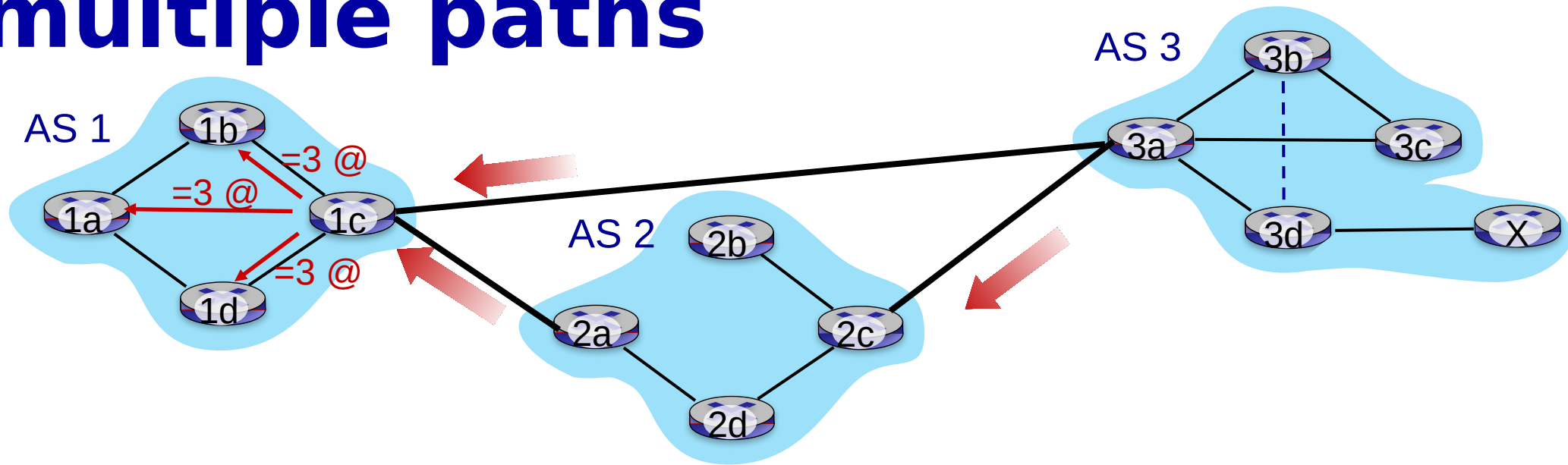
- BGP advertised route: prefix + attributes
 - prefix: destination being advertised
 - two important attributes:
 - **AS-PATH**: list of ASes through which prefix advertisement has passed
 - **NEXT-HOP**: indicates specific internal-AS router to next-hop AS
- **policy-based routing**:
 - gateway receiving route advertisement uses % \$ to accept/decline path (e.g., never route through AS Y).
 - AS policy also determines whether to " '* path to other neighboring ASes

BGP path advertisement



- AS2 router 2c receives path advertisement **AS3,X** (via eBGP) from AS3 router 3a
- based on AS2 policy, AS2 router 2c accepts path AS3,X, propagates (via iBGP) to all AS2 routers
- based on AS2 policy, AS2 router 2a advertises (via eBGP) path **AS2, AS3, X** to AS1 router 1c

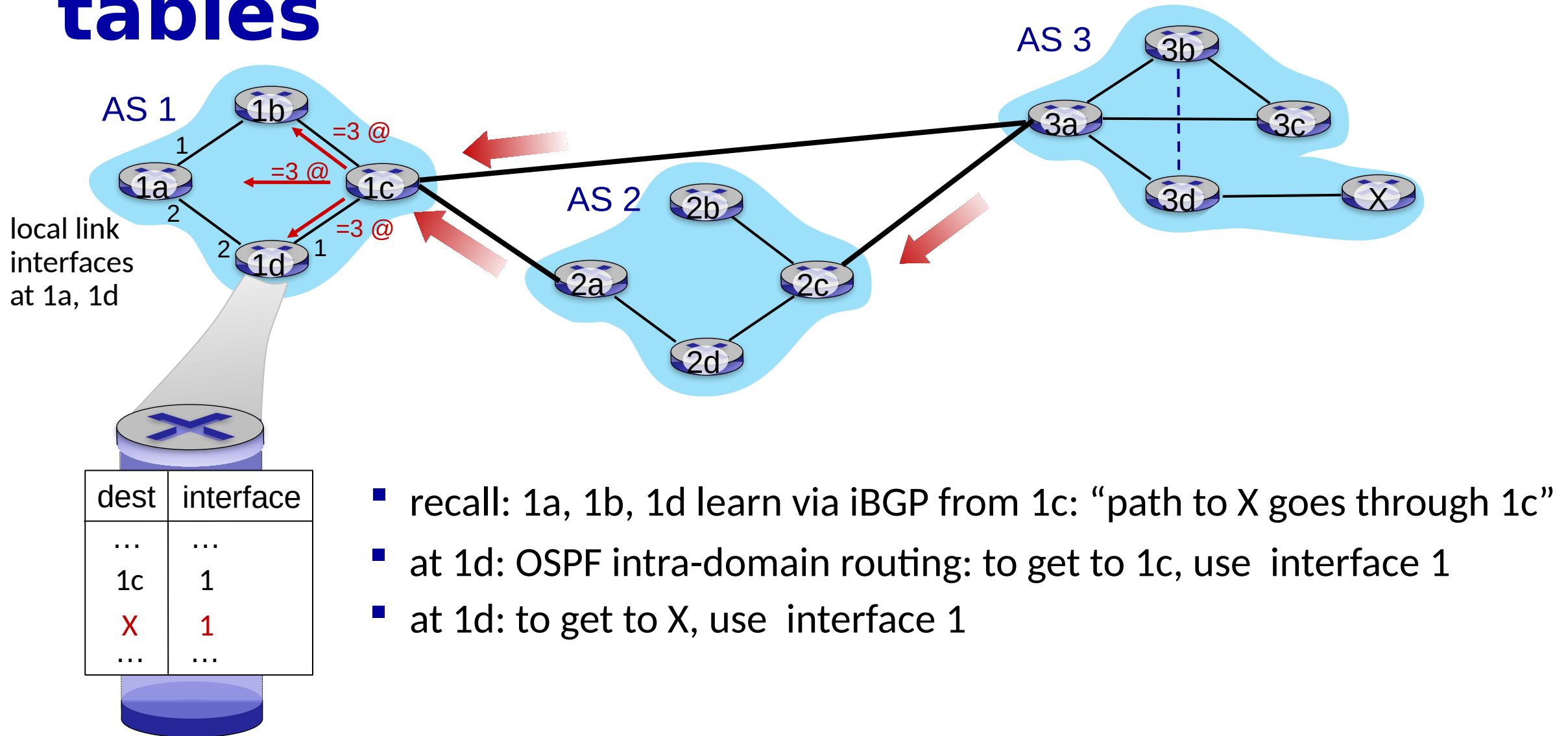
BGP path advertisement: multiple paths



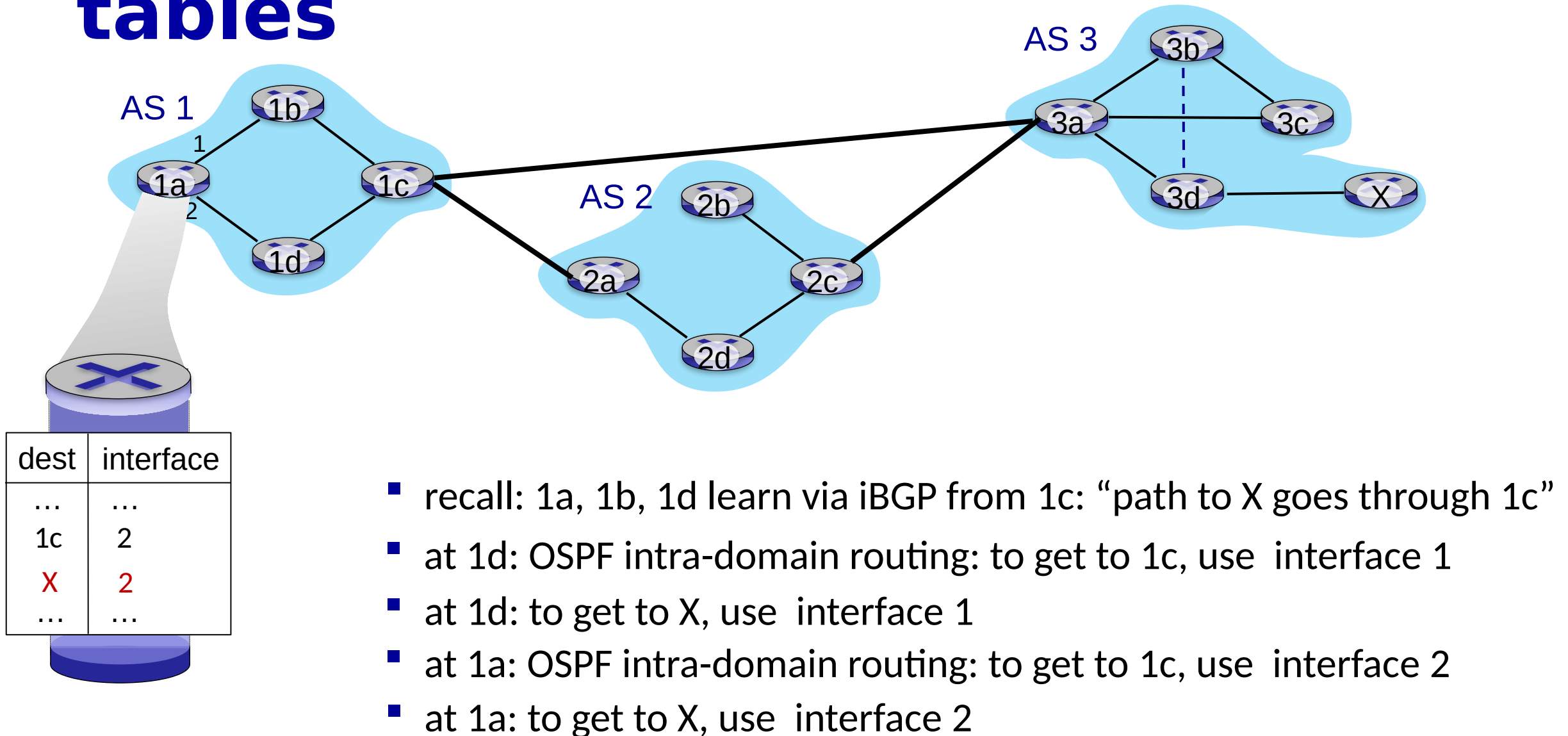
gateway router may learn about **multiple** paths to destination:

- AS1 gateway router 1c learns path $=0$ $=3$ @ from 2a
- AS1 gateway router 1c learns path $=3$ @ from 3a
- based on % \$ AS1 gateway router 1c chooses path $=3$ @ and advertises path within AS1 via iBGP

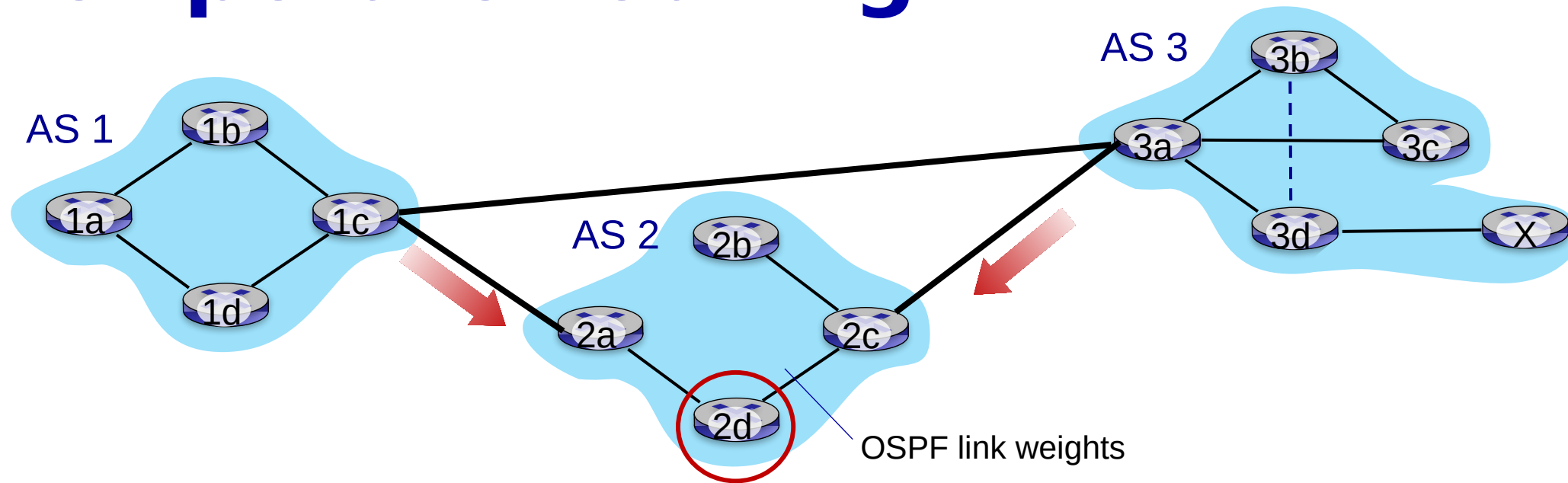
BGP: populating forwarding tables



BGP: populating forwarding tables

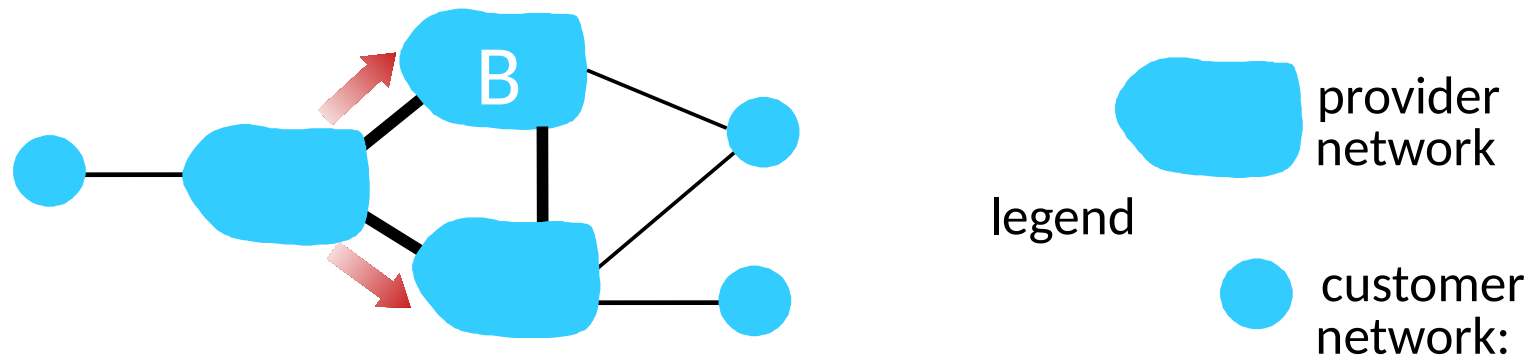


Hot potato routing



- 2d learns (via iBGP) it can route to X via 2a or 2c
- **hot potato routing**: choose local gateway that has least cost (e.g., 2d chooses 2a, even though more AS hops to X: don't worry about inter-domain cost!

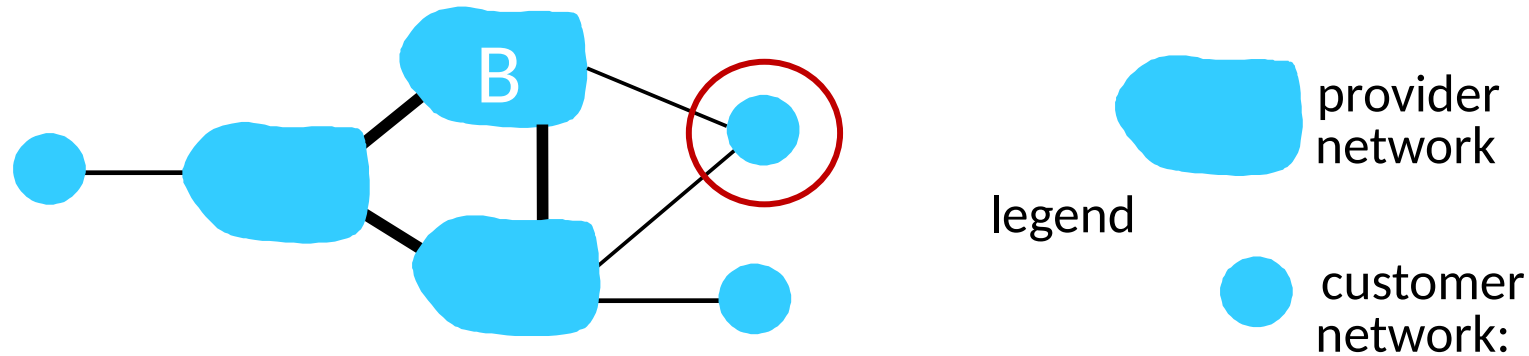
BGP: achieving policy via advertisements



ISP only wants to route traffic to/from its customer networks (does not want to carry transit traffic between other ISPs – a typical “real world” policy)

- A advertises path A_w to B and to C
- B * * " !* B A_w to C!
 - B gets no “revenue” for routing C B A_w, since none of C, A, w are B’s customers
 - C does learn about C B A_w path
- C will route C A_w (not using B) to get to w

BGP: achieving policy via advertisements (more)



ISP only wants to route traffic to/from its customer networks (does not want to carry transit traffic between other ISPs – a typical “real world” policy)

- A,B,C are **provider networks**
- x,w,y are **customer** (of provider networks)
- x is **dual-homed**: attached to two networks
- % \$ & x does not want to route from B to C via x
 - .. so x will not advertise to B a route to C

BGP route selection

- router may learn about more than one route to destination AS, selects route based on:
 1. local preference value attribute: policy decision
 2. shortest AS-PATH
 3. closest NEXT-HOP router: hot potato routing
 4. additional criteria

Why different Intra-, Inter-AS routing ?

policy:

- inter-AS: admin wants control over how its traffic routed, who routes through its network
- intra-AS: single admin, so policy less of an issue

scale:

- hierarchical routing saves table size, reduced update traffic

performance:

- intra-AS: can focus on performance
- inter-AS: policy dominates over performance